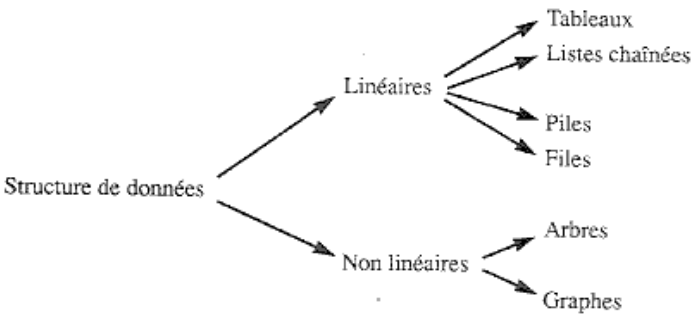
I – Introduction
II – Listes chaînées
1 – Liste chaînée simple
2 – Liste chaînée à en-tête
3 – Liste chaînée bidirectionnelle
III – Piles
IV – Files
1 – File simple
2 – File d'attente à deux queues
3 – File d'attente à niveau de priorité
V – Aperçu des structures non linéaires
1 – Arbres
2 – Graphes
VI – Allocation dynamique

4

Structures de données complémentaires

I – Introduction

Présentation des structures de données :



Les structures de données **LINEAIRES** possèdent une notion de séquence entre leurs éléments (premier, second, ..., suivant, ..., dernier).

– **Le tableau** : la séquence est donnée par les numéros de cases, (L'emplacement des éléments est linéaire).

– **La liste chaînée** : la séquence est donnée par une variable appelée POINTEUR, qui indique l'adresse où réside le suivant d'un élément. Cette structure ne fait jamais appel aux algorithmes de décalages vers la droite ou la gauche, c'est un avantage car le glissement des éléments est coûteux en temps d'exécution machine.

– **La pile** : cette structure restrictive de liste chaînée ne permet les ajouts et les suppressions qu'à une de ses extrémités appelée "sommet". C'est une structure de liste LIFO (Last In First Out), gérée comme une pile d'assiette, ajout au dessus, suppression au

dessus. Les piles sont couramment utilisées par le système d'exploitation, en particulier par l'unité arithmétique et logique lors de ses calculs.
– **La file** : cette structure restrictive de la liste chaînée ne permet les ajouts qu'à une de ses extrémités appelée "tête", et les suppressions à l'autre extrémité appelée "queue". C'est une structure de liste FIFO (First In First Out), gérée comme une file d'attente à un taxi, ajout à la fin de la file, suppression au début. Les files sont couramment utilisées par le système d'exploitation, par exemple lors de la gestion des programmes en attente du processeur central ou en attente de la disponibilité de l'imprimante.

Les structures de données **NON LINEAIRES** permettent de relier un élément à plusieurs suivants et non plus à un seul.
– **L'arbre** : cette structure associe 0, 1 ou 2 suivants à un élément. Elle permet, par exemple, d'effectuer des tris rapides.
– **Le graphe** : cette structure associe un ensemble d'éléments en créant un nombre quelconque de relations deux à deux. Elle permet, par exemple, d'effectuer des calculs d'optimisation dans l'enchaînement de travaux gérés par un PERT.

II – Listes chaînées

1 - LISTE CHAINÉE SIMPLE

DÉFINITION

Une liste est une collection linéaire d'éléments d'information (comporte un premier élément, un second ... et un dernier).

ex :	une liste d'achats	premier élément	–>	PAIN
		second	–>	BEURRE
		.....	–>	...
		dernier	–>	RIZ

REMARQUE

L'ordre des éléments d'information peut être traduit par le chaînage PHYSIQUE des cases d'un tableau (premier élément –> case n° 1, second –> case n° 2, etc.) mais cela pose des problèmes pour les insertions et les suppressions. Ceci nécessite des décalages vers la gauche à chaque suppression et vers la droite à chaque insertion, ces décalages augmentent le temps d'exécution machine.

AUTRE MÉTHODE

A chaque élément de la liste est associé un lien ou pointeur qui contient l'adresse de l'élément suivant de la liste. C'est la LISTE CHAINEE. Ceci peut se traduire en mémoire par

1 tableau des informations	INFO ( )
1 tableau des pointeurs	LIEN ( )
1 variable contenant l'adresse de la première information	DEBUT

Le pointeur du dernier élément contient une valeur particulière appelée pointeur nul (0

dans notre exemple). La liste vide a le pointeur NUL comme adresse de départ (variable DEBUT).

Exemple 1

Les noms des patients d'un hôpital sont stockés par ordre d'arrivée, un chaînage permet d'obtenir les noms par ordre alphabétique.

DEBUT = 3

ADR	INFO	LIEN
1	DUPONT	2
2	DURAND	6
3	ALBAN	1
4	ZOE	0
5	ROLAND	4
6	FLEUR	5

premier	:	ALBAN	à l'adresse	3	(début)
second	:	DUPONT	à l'adresse	1	(car lien(3)=1)
puis		DURAND		2	(car lien(1)=2)
		FLEUR		6	(car lien(2)=6)
		ROLAND		5	(car lien(6)=5)
		ZOE		4	(car lien(5)=4)

pas de suivant car lien(4)=0

Exemple 2

Les noms des 30 clients d'une banque et les noms des 4 responsables de comptes sont dans 2 tableaux. Un chaînage permet d'obtenir la liste des clients de chaque responsable.

ADR	CLIENT	LIEN-CLI
1	DUPOND	0
2	DURAND	6
3	ALBAN	22
4	ZOE	1
5	ROLAND	3
6	FLEUR	4
7	....	..
...	....	..
...	....	..

RESPONS	DEBUT-CLI
TOM	2
KIM	14
STAN	0
BOB	5

le responsable TOM gère les comptes de :
n° 2 : DURAND, n° 6 : FLEUR, n° 4 : ZOE et n° 1 : DUPOND

le responsable KIM gère les comptes de :
n° 14, etc.

le responsable STAN n'a aucun client

le responsable BOB gère les comptes de :
n° 5 : ROLAND, n° 3 : ALBAN, n° 22, etc.

DÉFINITION COMPLÉMENTAIRE

En même temps que la liste chaînée, il existe une liste particulière constituée des cases mémoire INUTILISEES. Cette liste qui possède son propre pointeur début est appelée liste de l'ESPACE DISPONIBLE ou de la MEMOIRE LIBRE. Le tableau des pointeurs LIEN ( ) contient le chaînage des places libres et la variable supplémentaire début-disponibilité est notée DEBDISP. Le dépassement de capacité est obtenu lorsque la variable DEBDISP est égale au pointeur NUL.

Exemple 3

L'exemple précédent des patients d'un hôpital évolue après le départ de certains malades, nous pouvons obtenir

	ADR	INFO	LIEN
	1	DUPONT	5
DEBUT = 6	2		0
	3	ALBAN	1
DEBDISP = 4	4		2
	5	ROLAND	0
	6	ABAR	3

la liste est : ABAR (6) ALBAN (3) DUPONT (1) ROLAND (5)

les disponibilités sont : cases 4 puis 2

remarque : il y a un pointeur NUL (fin de liste) pour chaque liste chaînée (celle des informations et celle des places disponibles).

❑ Exemple d'application 1

Soit la liste chaînée suivante :

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B			C	G		
LIEN	3	9	8	7	2	5	1	0	10	0
DEBUT		4								
DEBDISP		6								

1) Quelle est la chaîne des informations traitées ?
2) Quelle est la chaîne des cases disponibles ?
3) Quel chaînage a été utilisé ?

4) Que devient la liste après ajout de la lettre D ?
5) Que devient la liste après ajout de la lettre A ?
6) Que devient la liste après suppression de la lettre E ?
7) Que devient la liste après suppression de la lettre A ?

REMARQUE
– la liste de départ va évoluer au cours des questions 3, 4, 5 et 6
– pour ajouter, on utilise toujours la case n° debdisp
– une case dont le contenu est supprimé devient toujours première case disponible, soit debdisp

RÉPONSES

1) informations : B C E F G

2) disponibilités : 6 5 2 9 10

3) chaînage alphabétique

4)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B		D	C	G		
LIEN	3	9	8	7	2	1	6	0	10	0

DEBUT	4
DEBDISP	5

modifications de INFO(6), LIEN(6), LIEN(7), DEBDISP

5)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B	A	D	C	G		
LIEN	3	9	8	7	4	1	6	0	10	0

DEBUT	5
DEBDISP	2

modifications de INFO(5), LIEN(5), DEBUT, DEBDISP

6)

ADR	1	2	3	4	5	6	7	8	9	10
INFO			F	B	A	D	C	G		
LIEN	2	9	8	7	4	3	6	0	10	0

DEBUT	5
DEBDISP	1

modifications de LIEN(1), LIEN(6), DEBDISP

la modification de INFO(1) n'est pas obligatoire, car c'est le chaînage qui indique que l'adresse 1 est maintenant dans la chaîne des disponibilités et non des informations.

7)

ADR	1	2	3	4	5	6	7	8	9	10
INFO			F	B		D	C	G		
LIEN	2	9	8	7	1	3	6	0	10	0

DEBUT	4
DEBDISP	5

modifications de LIEN(5), DEBUT, DEBDISP

la modification de INFO(5) n'est pas obligatoire

■ Algorithme – exercice 1

Défilement a) afficher le contenu d'une liste

b) compter le nombre d'éléments dans une liste

Les algorithmes sont écrits sous forme de procédures paramétrées.

A) MÉTHODE

– déclaration d'un compteur cpt
– initialisation de cpt avec la variable début
– boucle de parcours de la liste avec affichage des informations et incrémentation du compteur cpt avec le contenu du tableau lien

```
procédure défilement (données: info, lien, début)
(* ce sous-programme affiche le contenu d'une liste *)
var cpt : entier
début
   afficher "voilà le contenu de la liste"
   cpt ← début
   tant que cpt  <> 0
     afficher info(cpt)
     cpt ← lien(cpt)
   ftq
   afficher "fin de liste"
fin
```

B) MÉTHODE

Nous devons ajouter à l'algorithme précédent :

– déclaration d'un compteur — `var totelt : entier`
– initialisation de ce compteur — `totelt ← 0`
– incrémentation de ce compteur (dans la boucle tant que) — `totelt ← totelt + 1`
– affichage du résultat

```
afficher "il y a ",totelt,
   " éléments dans la liste"
```

■ Algorithme – exercice 2

RECHERCHE a) dans une liste non ordonnée

b) dans une liste ordonnée (remarque: dichotomie impossible)

A) MÉTHODE

– test d'existence d'un élément dans la liste
– saisie de la valeur de l'information recherchée
– boucle de parcours des éléments de la liste et test d'égalité avec la valeur cherchée (mise à jour d'un booléen d'existence)
– affichage du résultat de la recherche

```
procédure recherche (données: info, lien, début)
(* la recherche est faite dans une liste non ordonnée *)
var cpt : entier
   elt : caractère (* en supposant que les informations traitées
                      sont de type caractère *)
   trouvé : booléen
début
   si    début <> 0
   alors afficher "donner la valeur de l'information recherchée"
         saisir elt
         (* début de recherche *)
         cpt ← début
         trouvé ← .faux.
         répéter
               si elt = info(cpt)
               alors trouvé ← .vrai.
               sinon cpt ← lien(cpt)
               fsi
         jqa cpt = 0 ou trouvé
                (* affichage résultat recherché *)
         si    trouvé
         alors afficher "cette information existe dans la liste"
         sinon afficher "cette information n'existe pas dans la liste"
         fsi
   sinon afficher "la liste est vide, recherche impossible"
   fsi
fin
```

REMARQUE

Attention : nous ne pouvons pas faire une boucle

```
répéter
....
jqa cpt = 0 où elt = info(cpt)
```

car lorsque cpt est égal à zéro, le contenu de la variable info(cpt) ou info(0) n'est pas défini.

B) MÉTHODE

Si la liste est ordonnée, nous optimisons le programme en prévoyant une sortie de boucle dès que la valeur cherchée est strictement inférieure à la valeur traitée dans la liste. (Utilisation d'un second booléen.)

```
(* le début de programme est identique *)
    (* début de recherche *)
    cpt ← début
    trouvé ← .faux.
    sortie ← .faux.
    répéter
        si    elt = info(cpt)
        alors trouvé ← .vrai.
        sinon si elt   <      info(cpt)
                alors sortie ← .vrai.
                sinon cpt ← lien(cpt)
                fsi
        fsi
    jqa cpt = 0 ou sortie ou trouvé
    (* suite du programme identique *)
```

■ Algorithme – exercice 3

INSERTION a) après un élément donné

b) en début de liste

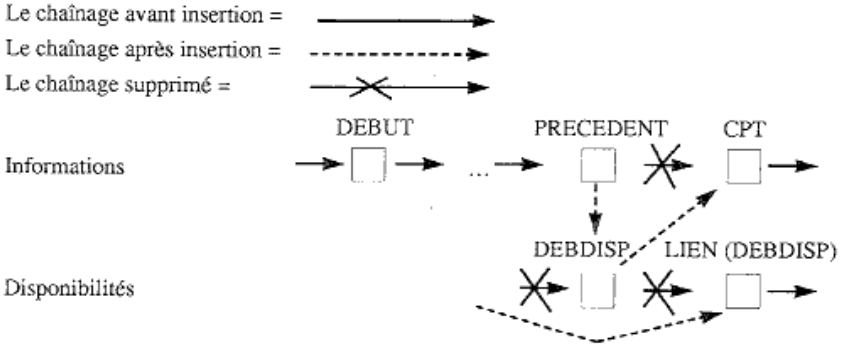
A) MÉTHODE :

– test sur l'existence d'espaces disponibles

– saisie de la valeur de l'information à insérer (à l'adresse debdisp) et de l'adresse de l'élément qui la précède

– mise à jour des 3 pointeurs suivants :

- la nouvelle disponibilité est le suivant de la disponibilité actuelle
- le nouveau est chaîné à l'actuel suivant du précédent
- le précédent est chaîné au nouveau




```
procédure insertion (données: début
                     données modifiées: info, lien, debdisp)
(* ce programme insère après un élément donné *)
var précédent, svgd : entiers
début
   si debdisp = 0
```

```
        alors afficher "liste pleine, aucune insertion possible"
        sinon afficher "donner la valeur de l'élément à insérer"
              saisir info(debdisp)
              afficher "donner le numéro de case de l'élément qui
                        doit précéder le nouveau"
              saisir précédent
              (* mise à jour des pointeurs *)
                      (* pointeur disponibilités *)
              svgd ← debdisp
              debdisp ← lien(debdisp)
                      (* pointeurs informations *)
              lien(svgd) ← lien(précédent)
              lien(précédent) ← svgd
        fsi
fin
```

REMARQUE

Attention à la permutation du contenu des trois cases :

debdisp, lien(debdisp) et lien(précédent)

il est NECESSAIRE d'utiliser une variable de sauvegarde (svgd)

B) MÉTHODE

Pour insérer au début de la liste, seule la mise à jour des pointeurs informations est modifiée, car il n'y a pas de précédent au premier élément, c'est la variable début qui précède le premier.

```
(* debut  de l'algorithme identique *)
(* mise à jour pointeurs *)
(* pointeur disponibilités *)
svgd ← debdisp
debdisp ← lien(debdisp)
(* pointeur informations *)
lien(svgd) ← début
début ← svgd
```

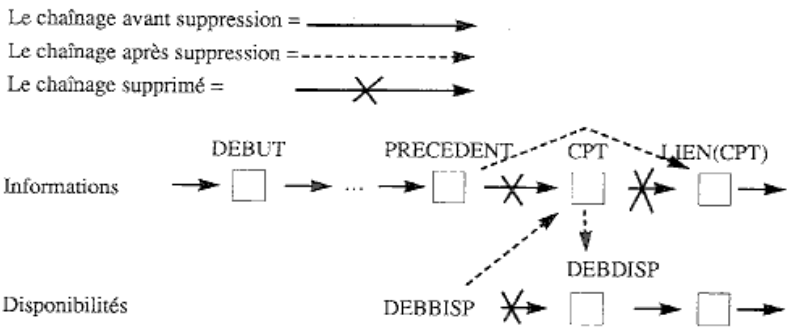
■ Algorithme – exercice 4

SUPPRESSION a) de l'élément qui suit un élément donné

b) du premier élément d'une liste

A) MÉTHODE

– test sur l'existence d'au moins un élément dans la liste
– saisie de l'adresse de l'élément qui précède celui à supprimer
– validation de la suppression
– mise à jour des 3 pointeurs suivants :
  • le précédent est chaîné au suivant de l'élément supprimé
  • la case supprimée est chaînée à la disponibilité actuelle
  • la nouvelle première disponibilité est la case supprimée



(la variable de sauvegarde pour la permutation est notée eltsupp)

```
procédure suppression
   (données: début
   données modifiées: info, lien, debdisp)
(* ce programme supprime l'élément qui suit un élément donné *)
var précédent, eltsupp : entier
    réponse : caractères
début
   si    début = 0
   alors afficher"liste vide, suppression impossible "
   sinon afficher"donner le numéro de case de l'élément
                qui précède celui à supprimer "
         saisir précédent
         afficher "l'élément à supprimer est : ",info(lien(précédent))
         afficher "validez-vous sa suppression (o/n) "
         saisir réponse
         si    réponse = "o"
         alors (* mise à jour des pointeurs *)
                     (* pointeur informations *)
               eltsupp ← lien(précédent)
               lien(précédent) ← lien(eltsupp)
                     (* pointeurs disponibilités *)
               lien(eltsupp) ← debdisp
               debdisp ← eltsupp
         fsi
   fsi
fin
```

B) MÉTHODE

Pour supprimer le premier élément d'une liste, seule la mise à jour des pointeurs est modifiée.

```
(* mise à jour des pointeurs *)
    (* pointeur information *)
svgd ← début
```

```
début ← lien(début)
     (* pointeurs disponibilités *)
lien(svgd) ← debdisp
debdisp ← svgd
```

■ Algorithme – exercice 5

CREATION DE LISTE

MÉTHODE

La création d'une liste se résume à une série d'ajouts, en début de liste, ou après le dernier élément de la liste, lorsque le chaînage a été bien initialisé. Cette initialisation consiste à considérer que la liste des informations est vide (début <– 0), et que la liste des disponibilités est maximale (debdisp vaut 1, et chaque case est chaînée avec sa suivante).

```
programme création
var info (1:10) : tableau de caractères
    lien (1:10) : tableau d'entiers
   début,debdisp,cpt : entiers
début
   (* initialisation *)
   début ← 0
   debdisp ← 1
   pour cpt ← 1 jqa 9
     lien(cpt) ← cpt + 1
   fpour
   lien(10) ← 0
   (* saisie des valeurs *)
        voir
             algorithme d'insertion en début de liste
        ou
             algorithme de défilement
                (mémorisant la dernière valeur du compteur cpt)
             et
             algorithme d'insertion après un élément donné (le dernier)
fin
```

■ PROBLEME 1

Soit la liste des 30 clients d'une banque et les noms des 4 responsables de comptes. Nous possédons un chaînage des clients par responsable de compte suivant l'ordre alphabétique des noms de clients. Les informations traitées sont :

- nom client
- solde compte chèque client
- solde compte épargne client
- nom responsable

Définir la structure des tables nécessaires, et écrire les procédures suivantes :

1) Edition des clients d'un responsable

2) Edition de tous les clients ayant un compte épargne (nous précisons le nom du client, le solde du compte épargne et le nom du responsable)

3) Ajout d'un client (nous saisissons les noms du responsable et du client et les deux soldes)

4) Suppression d'un client (nous saisissons le nom du client à supprimer)

SOLUTION :

ADR	NOM-CLIENT	SOLD-CHEQUE	SOLD-EPARGN	LIEN
1	DUPONT	500.50	0.00	7
2	DURAND	2546.37	842.50	8
3	ALBAN	- 350.00	0.00	1
4				13
5	ZOE	549999.00	6789.99	0
6				4
7	ROLAND	- 40.00	0.00	22
8	FLEUR	55555.55	111.10	5
9				
10	...	...	...	...

ADR	NOM-RESP	DEB-CLI
1	TOM	2
2	KIM	14
3	STAN	0
4	BOB	3

DEBDISP = 6

1) MÉTHODE

– saisie du numéro de responsable (entre 1 et 4)
– boucle de parcours de la liste de ce responsable

```
procédure édit1
   (données : nom-client, sold-chèque, sold-épargn, lien, nom-resp, deb-cli)
var noresp, cpt : entiers
début
   afficher "donner le n° de responsable"
   saisir noresp
   si noresp < 1 ou noresp > 4
   alors afficher "numéro de responsable incorrect"
   sinon afficher "responsable : " , nom-resp(noresp)
         afficher "nom des clients - solde chèque - solde épargne"
         cpt ← deb-cli(noresp)
         tant que cpt <> 0
            afficher nom-client(cpt), sold-chèque(cpt), sold-épargn(cpt)
            cpt ← lien(cpt)
         ftq
   fsi
fin
```

2) MÉTHODE

Pour accéder au nom du responsable, nous ne parcourons pas directement le tableau sold–épargn, car le nom du responsable reste introuvable quand le point de départ est une information client. Nous utilisons les informations des clients par l'intermédiaire du chaînage de la liste de chaque responsable.

– boucle de traitement des 4 représentants
– boucle de parcours de la liste d'un représentant et test d'existence d'une valeur au compte épargne. Affichage de ces valeurs.

```
procédure edit2 (données : nom-client,sold-chèque, sold-épargn,lien,
                 nom-resp,deb-cli)
var cpt,noresp : entiers
début
   pour noresp ← 1 jqa 4
     afficher "responsable : ",nom-resp(noresp)
     afficher " nom des clients - solde épargne "
     cpt ← deb-cli(noresp)
     tant que cpt <> 0
         si sold-épargn(cpt) > 0
         alors afficher nom-client(cpt),sold-épargn(cpt)
         fsi
         cpt ← lien(cpt)
     ftq
   fpour
fin
```

3) MÉTHODE

– test sur l'existence d'au moins une place disponible
– saisie du nom du responsable
– test sur l'existence du nom du responsable
– saisie du nom du client et des 2 soldes
– recherche de la place du nouveau dans le chaînage alphabétique
– mise à jour des pointeurs :
  • pointeurs des disponibilités
  • pointeurs des informations
  → dans le cas particulier d'un ajout au début de la liste
  → dans le cas d'un ajout après un élément donné

```
procédure ajout
  (données: nom-resp
  données modifiées: nom-client,sold-chèque,sold-épargn,lien,debdisp,deb-cli)
var  noresp,cpt,précédent,svgd : entiers
     nom : caractères
début
   si debdisp = 0
   alors afficher "liste pleine, pas d'ajout possible"
   sinon  (* recherche du nom du responsable dans la table *)
         afficher "donner le nom du responsable"
         saisir nom
         noresp ← 1
         tant que noresp < 4 et nom <> nom-resp(noresp)
              noresp ← noresp + 1
         ftq
         si nom <> nom-resp(noresp)
```

```
            alors afficher "responsable inexistant !!!"
            sinon (* traitement du client *)
                  afficher "donner le nom du client"
                  saisir nom-client(debdisp)
                  afficher "donner solde comptes chèque et épargne "
                  saisir sold-chèque(debdisp),sold-épargn(debdisp)
                     (* rechercher place dans chaînage alphabétique *)
                  cpt ← deb-cli(noresp)
                  sortie ← .faux.
                  répéter
                     si nom < nom-client(cpt)
                     alors sortie ← .vrai.
                     sinon précédent ← cpt
                           cpt ← lien(cpt)
                     fsi
                  jqa cpt = 0 ou sortie
                     (* mise à jour des pointeurs *)
                        (* pointeur disponibilités *)
                  svgd ← debdisp
                  debdisp ← lien(debdisp)
                        (* pointeurs informations *)
                  si    cpt = deb-cli(noresp)
                  alors    (* insertion au début *)
                     lien(svgd) ← deb-cli(noresp)
                     deb-cli(noresp) ← svgd
                  sinon    (* insertion après l'élément n° précédent *)
                     lien(svgd) ← cpt
                     lien(précédent) ← svgd
                  fsi
            fsi
      fsi
   fin
```

4) MÉTHODE

– saisie du nom du client à supprimer
– boucle de traitement des 4 représentants (pour la même raison qu'à la deuxième question, la mise à jour des listes par représentants reste impossible quand le point de départ est une information client)
– recherche du nom de client dans une des listes par représentants
– validation de la suppression
– mise à jour des pointeurs :
  • pointeurs des informations
    → dans le cas de la suppression du premier
    → dans le cas de la suppression de l'élément qui suit un élément donné
  • pointeurs des disponibilités

```
procédure suppression
     (données: nom-resp
     données modifiées: nom-client,sold chèque,
     sold-épargn,lien,deb-cli)
var  cpt, précédent,noresp : entiers
```

```
   nom : caractères
   trouvé : booléen
début
   afficher "donner le nom du client à supprimer"
   saisir nom
     (* parcours des listes des 4 responsables *)
   noresp ← 1
   trouvé ← .faux.
   tant que noresp <= 4 et non trouvé
        (* recherche du nom dans la liste ordonnée *)
     cpt ← deb-cli(noresp)
     sortie ← .faux.
     tant que cpt <> 0 et non sortie et non trouvé
         si nom = nom-client(cpt)
         alors trouvé ← .vrai.
         sinon si nom < nom-client(cpt)
               alors sortie ← .vrai.
               sinon précédent ← cpt
                     cpt ← lien (cpt)
               fsi
         fsi
     ftq
     si  trouvé
     alors afficher "suppression de ", nom, "soldes : ",
                    sold-chèque(cpt),sold-épargn(cpt)
          valid-maj (cpt,précédent,nom,noresp, lien,deb-cli,debdisp)
     fsi
     noresp ← noresp + 1
   ftq
   si non trouvé
   alors afficher "le client ", nom," n'est pas dans la liste"
   fsi
fin

procédure valid-maj
   (données: cpt,précédent,nom,noresp
   données modifiées: lien,déb-cli,debdisp)
var réponse : caractères
début
   afficher "validez-vous la suppression (oui/non) "
   saisir réponse
   si réponse = "oui"
   alors (* mise à jour pointeurs *)
              (* pointeur informations *)
         si cpt = deb-cli(noresp)
         alors (* suppression du premier *)
               deb-cli(noresp) ← lien(cpt)
         sinon (* suppression après n° précédent *)
               lien(précédent) ← lien(cpt)
         fsi
               (* pointeur disponibilités *)
```

```
            lien(cpt) ← debdisp
            debdisp ← cpt
            afficher "le client ",nom,"est supprimé"
        fsi
    fin
```

REMARQUE

Les instructions de recherche d'une place ou d'une valeur dans la liste chaînée sont identiques dans les algorithmes d'ajout et de suppression. Pour améliorer nos algorithmes, ces lignes doivent être mises en annexe dans un sous-programme. C'est ce que nous essayerons de faire par la suite.

2- LISTE CHAINÉE À EN-TETE

DÉFINITION

Une liste chaînée à en-tête est une liste chaînée qui contient toujours un élément particulier, l'élément d'en-tête, situé comme son nom l'indique en tête de la liste.

Il existe 3 types de liste chaînée à en-tête :
- Celles contenant un autre élément particulier de FIN DE LISTE (exemple 4).
- Celles dont le dernier élément pointe sur l'élément en-tête dite liste CIRCULAIRE (exemple 5).
- Celles qui sont circulaires et ont un élément de fin de liste (exemple 6).

Exemple 4 :

ADR	1	2	3	4	5	6	7	8	9	10
INFO	D	*		J	P	%		S		B
LIEN	4	10	7	5	8	0	0	6	3	1

DEBUT : 2 — DEBDISP : 9

- la liste est : B D J P S
- les cases disponibles sont : 9 3 7

INFO(2) = * est l'élément particulier en-tête
INFO(6) = % est l'élément particulier fin-de-liste

Remarque : la valeur de début est fixe, car même après des ajouts et des suppressions, début reste toujours égal à l'adresse de l'élément en-tête.

Exemple 5 :

ADR	1	2	3	4	5	6	7	8	9	10
INFO	D	*		J	P			S		B
LIEN	4	10	7	5	8	0	6	2	3	1

DEBUT : 2 — DEBDISP : 9

la liste est : B D J P S
les cases disponibles sont : 9 3 7 6

INFO(2) est l'élément particulier en-tête
LIEN(8) = 2 donc le dernier de la liste pointe sur l'élément en-tête

Le pointeur NUL n'est plus utilisé dans le chaînage des informations, mais il est conservé dans la liste des disponibilités.

Exemple 6 :

En remplaçant dans l'exemple 4, la valeur zéro de LIEN(6) par la valeur deux (qui est la valeur DEBUT) nous obtenons un exemple de liste chaînée circulaire avec un élément d'en tête et un élément fin de liste.

❑ Exemple d'application 2

Soit la liste alphabétique chaînée à EN-TETE CIRCULAIRE suivante :

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B			C	G		
LIEN	3	0	8	7	10	5	1	9	4	2

1) Quelles sont les valeurs associées de début et debdisp ?
   Trouvez la place de l'information "*".
2) Que devient la liste après suppression de la lettre B ?
3) Que devient la liste après ajout de la lettre A ?
4) Que devient la liste après ajout de la lettre J ?

RÉPONSES

1) DEBDISP = 6
   DEBUT = 9
   (l'élément en-tête est à l'adresse n° 9, car le dernier élément, la lettre G, possède un pointeur égal à 9)
   INFO(9) = "*"

2)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F				C	G	*	
LIEN	3	0	8	6	10	5	1	9	7	2

DEBUT = 9

DEBDISP = 4

modifications de LIEN(4), LIEN(9), DEBDISP

3)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	A			C	G	*	
LIEN	3	0	8	7	10	5	1	9	4	2

DEBUT = 9

DEBDISP = 6

modifications de INFO(4), LIEN(4), LIEN(9), DEBDISP

4)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	A		J	C	G	*	
LIEN	3	0	8	7	10	9	1	6	4	2

DEBUT = 9

DEBDISP = 5

modifications de INFO(6), LIEN(6), LIEN(8), DEBDISP

■ Algorithme – exercice 6

Réécrivons l'algorithme de recherche dans une liste non triée.

MÉTHODE

– Le booléen est maintenant inutile.
– L'initialisation se fait avec le contenu de lien(début) car l'élément d'en-tête ne contient pas une information de la liste réelle.
– Le test de fin de liste se fait en comparant cpt et début car le pointeur nul n'est plus pointeur du dernier élément de la liste.
– La liste vide est détectée lorsque le lien de début est égal à début.

```
procédure recherche   (données : info, lien, début)
(* ce programme effectue une recherche dans une liste chaînée à
en-tête circulaire *)
var  cpt : entier
     elt : caractères
début
   si lien(début) <> début
   alors afficher "donner la valeur de l'information recherchée"
        saisir elt
        (* début de recherche *)
        cpt ← lien(début)
        tant que cpt <> début et elt <> info(cpt)
             cpt ← lien(cpt)
        ftq
        si elt = info(cpt)
        alors afficher "cet élément existe dans la liste"
```

3)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	A			C	G	*	
LIEN	3	0	8	7	10	5	1	9	4	2

DEBUT = 9

DEBDISP = 6

modifications de INFO(4), LIEN(4), LIEN(9), DEBDISP

4)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	A		J	C	G	*	
LIEN	3	0	8	7	10	9	1	6	4	2

DEBUT = 9

DEBDISP = 5

modifications de INFO(6), LIEN(6), LIEN(8), DEBDISP

■ Algorithme – exercice 6

Réécrivons l'algorithme de recherche dans une liste non triée.

MÉTHODE

– Le booléen est maintenant inutile.
– L'initialisation se fait avec le contenu de lien(début) car l'élément d'en-tête ne contient pas une information de la liste réelle.
– Le test de fin de liste se fait en comparant cpt et début car le pointeur nul n'est plus pointeur du dernier élément de la liste.
– La liste vide est détectée lorsque le lien de début est égal à début.

```
procédure recherche   (données : info, lien, début)
(* ce programme effectue une recherche dans une liste chaînée à
en-tête circulaire *)
var  cpt : entier
     elt : caractères
début
   si lien(début) <> début
   alors afficher "donner la valeur de l'information recherchée"
        saisir elt
        (* début de recherche *)
        cpt ← lien(début)
        tant que cpt <> début et elt <> info(cpt)
             cpt ← lien(cpt)
        ftq
        si elt = info(cpt)
        alors afficher "cet élément existe dans la liste"
```

```
            sinon afficher "cet élément n'existe pas dans la liste"
            fsi
        sinon afficher "la liste est vide"
        fsi
    fin
```

INTÉRETS DE L'ÉLÉMENT D'EN-TETE

1) Les listes à en-tête permettent de simplifier les algorithmes, car, quel que soit l'élément d'une liste, l'adresse contenue dans son pointeur est toujours différente du pointeur NUL.

a) Le problème du test de INFO(cpt) quand cpt = 0 est donc supprimé.

(cf : remarque après l'algorithme - exercice 2 de recherche d'élément dans une liste chaînée)

L'algorithme de recherche dans une liste triée se simplifie aussi. Il est identique à l'algorithme précédent, sauf pour la boucle de recherche qui utilise le comparateur > au lieu de <>.

boucle : `tant que cpt <> début et elt > info(cpt)`

b) Lors d'une insertion ou d'une suppression, il n'est pas nécessaire de traiter le cas particulier du premier élément, car tout élément possède un précédent. Toutefois, il faut penser à initialiser précédent en même temps que cpt.

```
précédent ← début
cpt ← lien(début)
```

2) De plus, l'élément d'en-tête permet de stocker des informations complémentaires (exemple : le nombre total d'éléments contenus dans la liste).

ADR	NOM	SALAIRE	SEXE	LIEN	
1	DUPONT	3679	F	7	
2	ALBAN	12045	M	1	
3				6	DEBUT = 5
4	ROLAND	9867	F	8	
5	6	50351		2	DEBDISP = 3
6				9	
7	DURAND	10542	F	10	
8	ZOE	5430	M	5	
9				0	
10	FLEUR	8788	M	4	

Dans cette liste chaînée à en-tête circulaire, l'élément d'en-tête permet de stocker deux informations particulières, le nombre d'employés dans la variable nom(5) et le cumul des salaires dans la variable salaire(5).

■ PROBLEME 2

Soit la liste chaînée à en-tête circulaire donnée ci-dessus. Ecrire les procédures d'ajout et de suppression d'un employé.

1) MÉTHODE

Algorithme similaire à celui d'ajout dans une liste chaînée simple, avec la nouvelle procédure de recherche, écrite précédemment, et sans le traitement du cas particulier d'un ajout au début de la liste.

- test sur l'existence de places disponibles
- saisie des informations
- recherche de la place dans le chaînage
- mise à jour des pointeurs
- mise à jour des informations de l'élément en-tête

REMARQUE

Les instructions de recherche sont écrites dans un sous-programme pour pouvoir être exécutées par l'algorithme de suppression.

```
procédure ajout
   (données : début
   données modifiées : nom, salaire, sexe, lien, debdisp)
var cpt,précédent,svgd : entiers
début
   si debdisp = 0
   alors afficher "liste pleine, ajout impossible"
   sinon afficher "donner le nom, le salaire et le sexe de l'employé"
         saisir  nom (debdisp) ,salaire(debdisp) ,sexe(debdisp)
                 (* recherche place dans chaînage alphabétique *)
         recherche (nom(debdisp) ,nom,lien,début,cpt,précédent)
                 (* mise à jour des pointeurs *)
         svgd ← debdisp
         debdisp ← lien(debdisp)
         lien(précédent) ← svgd
         lien(svgd) ← cpt
               (* mise à jour des informations de l'élément en-tête *)
         nom(début) ← nom(début) + 1
         salaire(début) ← salaire(début) + salaire(svgd)
   fsi
fin

procédure recherche   (données : élément,nom,lien,début
                       résultat : cpt,précédent)
(* le chaînage alphabétique équivaut à un tri des informations
   de la liste, c'est donc une recherche dans une liste triée *)
début
   précédent ← début
   cpt ← lien(début)
   tant que cpt <> début et élément > nom(cpt)
     précédent ← cpt
     cpt ← lien(cpt)
   ftq
fin
```

2) MÉTHODE

– test sur l'existence d'au moins un élément dans la liste
– saisie du nom à supprimer
– recherche du nom dans la liste
– validation de la suppression
– mise à jour des pointeurs
– mise à jour des informations de l'élément en-tête

```
procédure suppression
   (données : début
   données modifiées : nom,salaire,sexe,lien,debdisp)
(* cette procédure utilise le même sous-programme de recherche
   que l'algorithme précédent *)
var cpt : entier
   réponse, nomsupp : caractères
début
   si    lien(début) = début
   alors afficher "liste vide, suppression impossible"
   sinon afficher "donner le nom à supprimer"
         saisir nomsupp
         recherche (nomsupp,nom,lien,début,cpt,précédent)
         si    nomsupp = nom(cpt)
         alors afficher "validez-vous la suppression (O/N)"
               saisir réponse
               si réponse = "O"
               alors (* mise à jour des pointeurs *)
                     lien(précédent) ← lien(cpt)
                     lien(cpt) ← debdisp
                     debdisp ← cpt
                     (* mise à jour élément en-tête *)
                     nom(début) ← nom(début) - 1
                     salaire(début) ← salaire(début) - salaire(cpt)
               fsi
         fsi
   fsi
fin
```

3 - LISTE CHAINÉE BIDIRECTIONNELLE

DÉFINITION

Une liste chaînée est bidirectionnelle si elle peut être parcourue dans les deux directions, du premier élément au dernier et réciproquement.

Une chaîne bidirectionnelle est composée des informations INFO, des pointeurs APRES des pointeurs AVANT, du pointeur PREMIER et du pointeur DERNIER.

Le pointeur APRES du dernier élément et le pointeur AVANT du premier élément contiennent tous les deux le pointeur NUL.

Exemple 7

(un changement de vocabulaire s'opère ici : le tableau lien est maintenant nommé APRES et la variable début est nommée PREMIER)

ADR	INFO	APRES	AVANT	
1	DUPONT	2	4	
2	FLEUR	6	1	
3		0		
4	ALBAN	1	7	PREMIER = 7
5		3		
6	ROLAND	0	2	DERNIER = 6
7	ABAR	4	0	DEBDISP = 5

la liste est :	ABAR	ALBAN	DUPONT	FLEUR	ROLAND
chaînage arrière :	ROLAND	FLEUR	DUPONT	ALBAN	ABAR
disponibilités :	5	3			

REMARQUE

La liste des DISPONIBILITES demeure MONODIRECTIONNELLE, elle utilise le pointeur nommé APRES pour son chaînage, le pointeur AVANT n'est pas mis à jour. Ceci s'explique par le fait que la liste des disponibilités n'est gérée que par insertion avant le premier (en cas de suppression dans la liste des informations) et suppression du premier (en cas d'insertion dans la liste des informations). Le chaînage arrière est donc inutile, il n'est jamais nécessaire de connaître le précédent d'un élément de la liste des disponibilités.

❑ Exemple d'application 3

Soit la liste chaînée bidirectionnelle suivante :

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B			C	G		I
APRES	3	9	8	7	2	5	1	10	0	0

1) Quelles sont les valeurs associées de avant, premier, dernier et debdisp ?
2) Que devient la liste après ajout de la lettre D ?
3) Que devient la liste après suppression de la lettre F ?

RÉPONSES

1)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B			C	G		I
APRES	3	9	8	7	2	5	1	10	0	0
AVANT	7	.	1	0	.	.	4	3	.	8

PREMIER = 4
DERNIER = 10
DEBDISP = 6

2)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E		F	B		D	C	G		I
APRES	3	9	8	7	2	1	6	10	0	0
AVANT	6	.	1	0	.	7	4	3	.	8

PREMIER = 4
DERNIER = 10
DEBDISP = 5
modifications de INFO(6), APRES(6), AVANT(6)
APRES(7), AVANT(1), DEBDISP

3)

ADR	1	2	3	4	5	6	7	8	9	10
INFO	E			B		D	C	G		I
APRES	8	9	5	7	2	1	6	10	0	0
AVANT	6	.	.	0	.	7	4	1	.	8

PREMIER = 4
DERNIER = 10
DEBDISP = 3
modifications de APRES(3), AVANT(3),
APRES(1), AVANT(8), DEBDISP

INTÉRETS DU CHAINAGE BIDIRECTIONNEL

Nous avons prouvé dans les algorithmes précédents la nécessité pour une suppression ou une insertion de connaître l'indice de l'élément qui précède l'élément concerné (indice stocké dans la variable précédent). Le chaînage arrière remplace cette étape de programmation. Il est surtout utile dans le cas du traitement de plusieurs chaînages.

Exemple :

Dans le problème de la liste chaînée des 30 clients d'une banque et des 4 responsables de comptes, gérons en plus le chaînage alphabétique de tous les clients tous représentants confondus. Lors de l'insertion ou de la suppression d'un élément, nous balayerons le chaînage monodirectionnel alphabétique des clients, et pour la mise à jour du chaînage bidirectionnel par représentants, nous utiliserons le pointeur avant de cette liste.

COMPLÉMENT

Nous pouvons aussi gérer des listes chaînées bidirectionnelles à EN-TETE avec les mêmes avantages qu'une monodirectionnelle.

Algorithme – exercice 7

DEFILEMENT
identique à l'algorithme de défilement dans une liste chaînée monodirectionelle, sans avantage particulier.

Algorithme – exercice 8

RECHERCHE
identique aussi. Dans le cas particulier d'une liste triée, la recherche d'un élément en fin de chaînage est optimisée en utilisant le pointeur DERNIER et les liens AVANT (balayage des éléments de la liste plus restreint).

Algorithme – exercice 9

INSERTION
il est inutile de sauvegarder l'adresse de l'élément précédant l'élément à insérer.
Le nombre de pointeurs à mettre à jour augmente.

MÉTHODE

Algorithme d'insertion dans une liste chaînée bidirectionnelle à en-tête circulaire, avec un élément fin de liste, pour éviter les cas particuliers déjà rencontrés.

– L'en-tête évite le cas particulier de l'insertion au début de la liste pour le chaînage APRES, PREMIER.

– L'élément fin de liste évite le cas particulier de l'insertion à la fin de la liste pour le chaînage AVANT, DERNIER.

– La solution circulaire évite d'utiliser des booléens pour ne pas tester l'information inexistante à l'adresse 0.

– test sur l'existence d'une place disponible
– saisie des informations
– recherche de la place dans le chaînage
– mise à jour des pointeurs

```
procédure insertion   (données : premier,dernier
                       données modifiées : info,après,avant,debdisp)
var cpt : entier
début
   si debdisp = 0
   alors afficher "liste pleine, ajout impossible"
   sinon afficher "donner l'information à insérer"
         saisir info(debdisp)
         recherche  (info(debdisp),info,après,premier,cpt)
         (* mise à jour des pointeurs *)
                 (* pointeur disponibilités *)
         svgd ← debdisp
         debdisp ← après(debdisp)
                 (* pointeur après *)
```

```
            après (avant(cpt)) ← svgd
            après (svgd) ← cpt
            (* pointeur avant *)
            avant (svgd) ← avant(cpt)
            avant (cpt) ← svgd
      fsi
fin


procédure recherche    (données : élément,info,lien,début
                        résultat : cpt)
début
   cpt ← lien(début)
   tant que cpt <> début et élément > info(cpt)
     cpt ← lien(cpt)
   ftq
fin
```

Algorithme – exercice 10

SUPPRESSION

il est inutile de sauvegarder l'adresse de l'élément précédant l'élément à supprimer.
Le nombre de pointeurs à mettre à jour augmente.

MÉTHODE

Algorithme de suppression dans une liste chaînée bidirectionnelle à en-tête circulaire, avec un élément fin de liste, pour éviter les cas particuliers connus.

- test sur l'existence d'au moins un élément dans la liste
- saisie de l'information
- recherche de la place de l'information dans la liste
- validation de la suppression
- mise à jour des pointeurs

```
procédure suppression (données : premier,dernier
                       données modifiées : info,après,avant,debdisp)
var cpt : entier
   réponse, infosupp : caractères
début
   si premier = 0
   alors afficher "liste vide, suppression impossible"
   sinon afficher "donner la valeur de l'information à supprimer"
         saisir infosupp
         recherche (infosupp,info,après,premier,cpt)
         si    infosupp = info(cpt)
         alors afficher "validez-vous la suppression (o/n)"
               saisir réponse
               si réponse = "o"
```

```
                    alors (* mise à jour des pointeurs *)
                          (* pointeur après *)
                          après(avant(cpt)) ← après(cpt)
                          (* pointeur avant *)
                          avant(après(cpt)) ← avant(cpt)
                          (* pointeur disponibilités *)
                          après(cpt) ← debdisp
                          debdisp ← cpt
                    fsi
              fsi
        fsi
    fin
```

CONCLUSION

La chaîne bidirectionnelle est peu utilisée, car elle est plus couteuse en temps d'exécution machine du fait de la mise à jour des nombreux pointeurs.

III – Les piles

DEFINITION

Une pile est une liste d'informations telle qu'un élément ne peut être ajouté ou retiré que par une extrémité appellée SOMMET de la pile.

Terminologie particulière aux piles :

PUSH = empiler = adjonction d'un élément
POP = dépiler = suppression d'un élément

REMARQUE

La liste des disponibilités déjà décrite est une pile. (Le dernier élément entré est le premier à sortir.)

Exemple

L'appel en cascade de plusieurs sous-programmes est géré par une pile. Si le programme A appelle B qui appelle C qui appelle D, la pile des programmes en attente évoluera comme suit :

				C			
			B	B	B		
PILE		A	A	A	A	A	
temps	T0	T1	T2	T3	T4	T5	T6

T0 : lancement du programme A
T1 : appel du programme B par A

T2 : appel du programme C par B
T3 : appel du programme D par C
T4 : poursuite du programme C (quand D est terminé)
T5 : poursuite du programme B (quand C est terminé)
T6 : poursuite du programme A (quand B est terminé)

REPRÉSENTATION

On peut représenter une pile par un tableau linéaire PILE avec deux variables associées, le point de remplissage de la pile noté SOMMET et la taille maximale de la pile notée MAXPILE.

❑ Exemple d'application 4

Soit la pile suivante :

ADR	1	2	3	4	5	6	7	8	9	10
PILE	X	Z	G	T	A					

1) Quelles sont les valeurs des variables sommet et maxpile ?
2) Que devient la pile après ajout de la lettre B ?
3) Que devient la pile après ajout de la lettre S ?
4) Que devient la pile après suppression d'une lettre ?

RÉPONSES :

1) SOMMET = 5
MAXPILE = 10

2)

ADR	1	2	3	4	5	6	7	8	9	10
PILE	X	Z	G	T	A	B				

SOMMET = 6
MAXPILE = 10

3)

ADR	1	2	3	4	5	6	7	8	9	10
PILE	X	Z	G	T	A	B	S			

SOMMET = 7
MAXPILE = 10

4)

ADR	1	2	3	4	5	6	7	8	9	10
PILE	X	Z	G	T	A	B				

SOMMET = 6
MAXPILE = 10

Par définition de la pile, c'est le dernier élément ajouté qui est retiré.

REMARQUE

La variable maxpile détermine la taille de la zone mémoire réservée à la pile. L'attribution

d'une valeur à cette variable est complexe :
– un large espace est coûteux en place mémoire
– un faible espace est coûteux en temps de gestion des dépassements

Une solution existe pour traiter deux piles et minimiser les dépassements de capacité : un tableau unique.

ADR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
PILE	Z	F	U	M	D									R	C	A

SOMMET1 = 5 SOMMET2 = 14

MAXPILE1 = SOMMET 2 - 1 MAXIPILE2 = SOMMET1 + 1

La pile P1 évolue de la gauche vers la droite, son sommet croît.

La pile P2 évolue de la droite vers la gauche, son sommet décroît.

■ Algorithme – exercice 11

ADJONCTION d'un élément à la pile (PUSH)

MÉTHODE
– test sur l'existence d'une place libre
– mise à jour de la variable sommet
– stockage de l'élément ajouté dans la pile

```
procédure push   (données : élément,maxpile
                  données modifiées : pile,sommet)
début
   si sommet < maxpile
   alors sommet ← sommet + 1
         pile(sommet) ← élément
   sinon afficher "pile pleine, empilement impossible"
   fsi
fin
```

■ Algorithme – exercice 12

SUPPRESSION d'un élément de la pile (POP).

MÉTHODE

Nous remarquons que le fait de dépiler ne rend plus accessible un élément, sans pour autant le détruire.
– test sur l'existence d'un élément dans la pile
– récupération de l'élément supprimé de la pile
– mise à jour de la variable sommet

```
procédure pop (données : pile
               données modifiées : sommet
               résultat : élément)
début
   si sommet > 0
   alors élément ← pile(sommet)
         sommet ← sommet   1
   sinon afficher "pile vide, dépilement impossible"
   fsi
fin
```

REMARQUE

Les deux exercices qui suivent permettent de comprendre le fonctionnement interne de l'ordinateur, ils expliquent le mécanisme de traitement de l'unité arithmétique et logique lors d'un calcul.

■ Algorithme – exercice 13

CONVERSION d'un nombre exprimé en base 10 en base b.

MÉTHODE

Les différents restes des divisions successives des quotients par la base sont empilés. Le résultat apparaît correctement lorsque nous effectuons les dépilements.

Exemple

11 en base 2		quotient	reste					
		11	1					
		5	1					
		2	0					
		1	1					
		0						
			1					
			0	0				
		1	1	1				
PILE	1	1	1	1	Résultat : 1	0	1	1

– initialisation de la pile
– saisie du nombre à convertir et de la base
– calculs et gestion de la pile
– affichage du résultat

```
programme conversion
const maxpile c'est 50
var pile (1:maxpile) : tableau d'entiers
    sommet,maxpile : entiers
```

```
    nombre,base,valeur,svgd : entiers
début
        (* initialisation de la pile *)
    sommet ← 0
        (* saisie *)
    afficher "donner le nombre à convertir et la base de conversion"
    saisir nombre,base
    svgd ← nombre
        (* calcul *)
    tant que nombre >  0
      push( nombre MOD base, maxpile, pile, sommet)
      nombre ← nombre Div base
    ftq
        (* affichage *)
    afficher "le nombre ",svgd," en base ",base, " = "
    tant que sommet > 0
      pop ( pile,sommet,valeur)
      afficher valeur
    ftq
fin
```

■ Algorithme – exercice 14

CALCUL NUMERIQUE

– traduction d'une expression mathématique en notation polonaise suffixée
– évaluation de l'expression exprimée en notation polonaise suffixée

REMARQUE

Ceci explique le fonctionnement réel de l'unité arithmétique et logique.

– notation courante	A + B	C – D	E * F	G / H
polonaise préfixée	+ A B	– C D	* E F	/ G H
polonaise suffixée	A B +	C D –	E F *	G H /

Exemple

4 + 3	s'écrit	4 3 +
(2 + 5) * 4	s'écrit	(2 5 +) * 4
	soit	2 5 + 4 *
(5 / 8) ^ 2 – (6 + 23)		
	s'écrit	(5 8 /) ^ 2 – (6 23 +)
	soit	5 8 / 2 ^ – (6 23 +)
	soit	5 8 / 2 ^ 6 23 + –

le ^ est le signe de puissance.

(2 – 17) * (3 / (10 – 5))		
	s'écrit	(2 17 –) * (3 / (10 5 –))
	soit	(2 17 –) * (3 10 5 – /)
	soit	2 17 – 3 10 5 – / *

Entraînez-vous sur de nombreux exemples.

MÉTHODE POUR L'ÉVALUATION

ex : 5 * (6 + 2) – 12 / 4 ——> 5 6 2 + * 12 4 / –

Le travail consiste à traiter tous les caractères dans l'ordre de la notation polonaise suffixée jusqu'au dernier. Deux cas sont rencontrés :
– nous traitons un nombre ——> il faut l'empiler
– nous traitons un opérateur de calcul ——> il faut dépiler deux nombres
effectuer le calcul
empiler le résultat.

La valeur de l'expression calculée est le dernier élément qui se trouve dans la pile.

Notre exemple fera évoluer la pile comme suit :

			2	]				4	]			
		6	6	]	8	]		12	12	]	3	]
PILE	5	5	5		5	]	40	40	40		40	] 37
opérations					+		*				/	–

lexique :

pile, sommet, maxpile : éléments de gestion de pile
exp : tableau de l'expression mathématique
polo : tableau de la traduction en notation polonaise
lgpolo : nombre de terme de l'expression en notation polonaise

```
procédure évaluation  (données : polo, lgpolo, maxpile
                       données modifiées : pile, sommet)
var  cpt,nb1,nb2,code,indice : entiers
   result : réel
début
   pour cpt ← 1 jqa lgpolo
     si rang ("^*/+-",polo(cpt)) = 0
     alors (* nous traitons un nombre *)
         push (polo(cpt),maxpile,pile,sommet)
     sinon     (* nous traitons un opérateur *)
         pop ( pile, sommet, nb2 )
         pop ( pile, sommet, nb1 )
         suivant polo(cpt)    faire
            "^"   :  result ← nb1 ^nb2
            "*"   :  result ← nb1 * nb2
            "/"   :  si nb2 = 0
                     alors afficher "division par zéro impossible"
                     sinon result ← nb1 / nb2
```

```
                    fsi
                "+" : result ← nb1 + nb2
                "-" : result ← nb1 - nb2
            fsuivant
                (* nous empilons le résultat *)
            push ( result, maxpile, pile, sommet)
        fsi
    fpour
        (* affichage du résultat *)
    pop ( pile, sommet, résult)
    afficher "le résultat est : ", résult
fin
```

MÉTHODE POUR LA TRADUCTION :

Nous supposons qu'il n'y a que les signes : ^ * / + – ( )
avec l'ordre de priorité connu et l'exécution de gauche à droite. Nous empilons une parenthèse ouvrante "(" au départ de la pile et nous ajoutons une parenthèse fermante ")" à la fin de l'expression.

Le travail consiste à traiter les caractères dans l'ordre de l'expression mathématique, jusqu'à la fin. Quatre cas se présentent :

– nous traitons un nombre ——> il va compléter le résultat
– nous traitons une "(" ——> nous l'empilons
– nous traitons une ")" ——> nous dépilons jusqu'à "(" comprise.
Les opérateurs dépilés vont compléter le résultat.
– nous traitons un opérateur ——> nous dépilons tous les opérateurs plus prioritaires ou de priorité égale, et les ajoutons au résultat. Puis nous empilons l'opérateur traité.

Notre exemple fera évoluer la pile comme suit :

caractère traité :	5	*	(	6	+	2	)	–	12	/	4	)
					+							
			(		(		(			/		
		*	*		*		*	*	–	–	–	
PILE	(		(	(		(		(	(	(	(	(
résultat :		5			6		2	+	*		12	4	/	–

REMARQUE

Le calcul de priorité est fait en attribuant la priorité 0 à la puissance, 1 à la multiplication et la division, 2 à l'addition et la soustraction. Cette attribution est faite par calcul de la position de l'opérateur dans la chaîne "^*/+–", position = 1, 2, 3, 4 ou 5 divisée par 2.

RANG ("^*/+-",elt) DIV 2
= 0 pour ^
= 1 pour * et /
= 2 pour + et -

Lexique complémentaire :

cpt 1 : indice de parcours du tableau exp
cpt 2 : indice de parcours du tableau polo
prior1 : priorité d'un élément de exp
prior2 : priorité d'un élément dépilé
Lgexp : nombre de termes de l'expression mathématique.

```
procédure traduction  (données : exp,maxpile
                       données modifiées : pile,sommet, Lgexp
                       résultat : polo,Lgpolo)
var elt : caractères
   cpt1, cpt2, prior1, prior2 : entiers
début
     (* initialisation de la pile *)
   sommet ← 0
     (* empilement parenthèse ouvrante au départ *)
   push ("(")
     (* ajout parenthèse fermante à la fin de l'expression *)
   lgexp ← lgexp + 1
   exp(lgexp) ← ")"
     (* traitement *)
   cpt 2 ← 1
   pour cpt1 ← 1 jqa Lgexp
     suivant exp(cpt1) faire
         "(" : push ( "(", maxpile, pile, sommet)
         ")" : pop ( pile, sommet,elt)
               tant que elt <> "("
                     polo(cpt2) ← elt
                     cpt2 ← cpt2 + 1
                     pop ( pile, sommet, elt)
               ftq
         "^","*","/","+","-" :
               (* il faut dépiler les opérateurs plus
               prioritaires ou de priorité égale *)
               (* priorité de l'opérateur traité *)
               prior1 ← RANG ("^*/+-",exp(cpt1)) DIV 2
               (* dépilement et calcul priorité *)
               pop ( pile, sommet, elt)
               prior2 ← RANG ("^*/+-",elt) DIV 2
               (* boucle de traitement *)
               tant que prior2 <= prior1
                     polo(cpt2) ← elt
                     cpt2 ← cpt2 + 1
                     pop ( pile, sommet, elt)
                     prior2 ← RANG ("^*/+-",elt) DIV 2
                  ftq
                     (* nous rempilons l'élément non prioritaire *)
                  push ( elt , maxpile, pile, sommet)
                     (* maintenant empilons l'opérateur traité *)
                  push ( exp(cpt1), maxpile, pile, sommet)
```

```
            sinon : (* c'est un nombre, il faut le stocker *)
                    polo(cpt2) ← exp(cpt1)
                    cpt2 ← cpt2 + 1
        fsuivant
    fpour
          (* longeur de l'expression en notation polonaise *)
    Lgpolo ← cpt2 - 1
fin
```

IV – Les files

1 - FILE SIMPLE

DÉFINITION

Une file d'attente est une liste d'éléments telle que les suppressions ne peuvent être effectuées qu'à une de ses extrémités appelée TETE, les insertions à l'autre extrémité appelée QUEUE.

Représentation :

On peut représenter la file comme un tableau linéaire FILE avec deux variables, TETE et QUEUE, qui indiquent les numéros de cases des éléments extrêmes de la file, et une variable MAXFILE fixant la taille du tableau. Si TETE = pointeur NUL alors la file est vide.

Exemple 8

ADR	1	2	3	4	5	6	7	8	9	10
PILE	X	T	D	J	I	Z				

TETE = 1 QUEUE = 6 MAXFILE = 10

REMARQUE :

Ajoutons les 4 lettres suivantes : U, C, O, P à la file donnée en exemple, le pointeur QUEUE prend pour valeur 10, supprimons 3 éléments en tête de file (les lettres X, T, puis D), le pointeur TETE prend pour valeur 4. On obtient :

ADR	1	2	3	4	5	6	7	8	9	10
PILE				J	I	Z	U	C	O	P

TETE = 4 QUEUE = 10 MAXFILE = 10

Nous avons QUEUE = MAXFILE, et pourtant, il y a de la place libre dans le tableau. Il existe deux solutions pour permettre encore des ajouts :

1) Décaler les éléments vers la gauche. Cette solution n'est pas retenue car elle est bien trop coûteuse en temps d'exécution machine.

2) Considérer que le tableau est circulaire :
- l'ajout qui suit l'ajout en dernière case se fait en première case si elle est libre.
  Si queue = maxfile, prochain ajout queue = 1
- la suppression qui suit celle de la dernière case concerne la première.
  Si tête = maxfile, prochaine suppression tête = 1
- la suppression du dernier élément provoque TETE = QUEUE = NUL.

❑ Exemple d'application 5

Soit la file suivante :

ADR	1	2	3	4	5	6	7	8	9	10
PILE				T	A	R	L	V		

1) Quelles sont les valeurs des variables tête et queue associées ?
2) Que devient la file après ajout de la lettre N ?
3) Que devient la file après suppression d'un élément ?
4) Que devient la file après ajout de la lettre H ?
5) Que devient la file après ajout de la lettre K ?
6) Que devient la file après suppression de 6 éléments ?
7) Que devient la file après suppression d'un élément de plus ?

RÉPONSES :

1) TETE = 4
   QUEUE = 8

2)

ADR	1	2	3	4	5	6	7	8	9	10
PILE				T	A	R	L	V	N	

TETE = 4
QUEUE = 9

3)

ADR	1	2	3	4	5	6	7	8	9	10
PILE					A	R	L	V	N	

TETE = 5
QUEUE = 9

4)

ADR	1	2	3	4	5	6	7	8	9	10
PILE					A	R	L	V	N	H

TETE = 5
QUEUE = 10

5)

ADR	1	2	3	4	5	6	7	8	9	10
PILE	K				A	R	L	V	N	H

TETE = 5

QUEUE = 1

6)

ADR	1	2	3	4	5	6	7	8	9	10
PILE	K									

TETE = 1

QUEUE = 1

7)

ADR	1	2	3	4	5	6	7	8	9	10
PILE										

TETE = 0

QUEUE = 0

Les informations supprimées de la file n'apparaissent plus dans les exemples pour faciliter la compréhension, mais physiquement les cases ne sont pas nettoyées de leur contenu, seuls les pointeurs TETE et QUEUE valident le contenu d'une case.

■ Algorithme – exercice 15

AJOUT d'un élément à la file

MÉTHODE

– recherche du numéro de case prévu pour cet ajout
– test de vérification de la disponibilité de cette case
– stockage de l'élément dans la file
– mise à jour de la variable queue
– mise à jour de la variable tête dans le cas particulier de l'ajout du premier élément de la file

```
procédure ajout (données : élément, maxfile
                 données modifiées : file, tête, queue)
var place : entier
début
     (* recherche place = n° case pour prochain ajout *)
   si queue = maxfile
   alors place ← 1
   sinon place ← queue + 1
   fsi
     (* test tableau plein *)
   si    place = tête
   alors afficher "file pleine, ajout impossible"
   sinon file(place) ← élément
         queue ← place
         si tête = 0
         alors (* premier ajout dans une file vide *)
               tête ← 1
```

```
            fsi
      fsi
fin
```

■ Algorithme – exercice 16

SUPPRESSION d'un élément de la file

MÉTHODE
- test sur l'existence d'un élément dans la file
- stockage dans une variable de l'élément supprimé
- mise à jour du pointeur tête
  - → dans le cas où l'élément à supprimer est en dernière case
  - → dans le cas où l'élément à supprimer est le dernier de la file
  - → dans le cas général (tête pointe sur la case suivante)

```
procédure suppression (données : maxfile, file
                       données modifiées : tête, queue
                       résultat : élément)
début
   si tête = 0
   alors afficher "file vide, suppression impossible"
   sinon élément ← file(tête)
         si    queue = tête
         alors    (* suppression du dernier *)
               queue ← 0
               tête ← 0
         sinon tête ← tête + 1
               si tête > maxfile
               alors tête ← 1
               fsi
         fsi
   fsi
fin
```

2 - FILE D'ATTENTE À DEUX QUEUES : DEQUE

DÉFINITION :

"Double-Ended-Queue" : file d'attente à 2 queues, ajouts et suppressions sont permis à l'une et à l'autre extrémité mais jamais au milieu.

Exemple :

Les personnes prioritaires sont ajoutées en début et non en fin de la file d'attente, nous remarquons qu'il n'y a, dans cet exemple, qu'une seule extrémité de retrait.

■ Algorithme – exercice 17

Ajout et suppression dans une "deque". Pour rappeler les algorithmes précédemment écrits, nous saisissons les éléments à ajouter ou à supprimer, accompagnés d'un code indiquant si l'action se situe en tête de file ou en queue.

```
procédure majdeque (données : maxfile, file, tête, queue)
var choix, rep, elt : caractères
début
   afficher "(A)jout ou (S)uppression ? "
   saisir choix
   afficher "en (T)ête ou (Q)ueue de file ? "
   saisir rep
   si choix = "A"
   alors afficher "donner l'élément à ajouter"
        saisir elt
        si    rep = "Q"
        alors ajout (elt,maxfile,file,tête,queue)
        sinon ajout-tête (elt,maxfile,file,tête,queue)
        fsi
   sinon si rep = "T"
        alors suppression (maxfile,file,tête,queue,elt)
        sinon suppr-queue (maxfile,file,tête,queue,elt)
        fsi
   fsi
fin
```

- les deux sous-programmes ajout-tête et suppr-queue ont des méthodes similaires aux sous-programmes ajout et suppression précédemment écrits.

```
procédure ajout-tête
   (données : élément maxfile,
   données modifiées: file, tête, queue)
var place : entier
début
     (* recherche place *)
   si tête = 1 ou tête = 0
   alors place ← maxfile
   sinon place ← tête - 1
   fsi
     (* test tableau plein *)
   si place = queue
   alors afficher "file pleine, ajout impossible"
   sinon file(place) ← élément
        tête ← place
        si queue = 0
        alors       (* premier ajout en tête dans une file vide *)
                queue ← maxfile
        fsi
   fsi
fin

procédure suppr-queue
   (données : maxfile,file
   données modifiées: , tête, queue
   résultat : élément)
début
   si tête = 0
   alors afficher "file vide, suppression impossible"
```

```
        sinon élément ← file(queue)
              si queue = tête
              alors (* suppression du dernier *)
                    queue ← 0
                    tête ← 0
              sinon si queue > 1
                    alors queue ← queue - 1
                    sinon queue ← maxfile
                    fsi
              fsi
        fsi
  fin
```

3 - FILE D'ATTENTE À NIVEAU DE PRIORITÉ

DÉFINITION

Ensemble d'éléments tels qu'un niveau de priorité est affecté à chacun. Nous les extrayons dans l'ordre des valeurs de priorités croissantes (le niveau le plus élevé est représenté par la valeur de priorité la plus faible), et dans le cas de priorité égale dans l'ordre d'insertion.

Représentations

Deux représentations sont possibles :

– Utilisation d'une liste chaînée monodirectionnelle (exemple 9):
le rangement des tableaux INFO, PRIORITE et LIEN est tel que
X précède Y si valeur priorité (X) < valeur priorité (Y)
ou si à priorités égales, X a été inséré avant Y

– Utilisation de plusieurs files (exemple 10):
une file d'attente est attribuée à chaque niveau de priorité (chacune avec son propre tableau circulaire FILE et ses pointeurs TETE et QUEUE). Nous remarquons que le tableau FILE peut être de dimension 2 et les pointeurs stockés dans un tableau de dimension 1.

Exemple 9

ADR	INFO	PRIORITE	LIEN
1	P2	0	3
2			9
3	P5	1	8
4	P8	2	7
5			2
6	P6	4	10
7	P4	4	6
8	P1	1	4
9			0
10	P3	4	0

DEBUT = 1

DEBDISP = 5

La liste est P2 P5 P1 P8 P4 P6 P3

Les programmes correspondants seront donc exécutés dans cet ordre.

Priorité 0 (pour P2), puis 1 (pour P5 et P1), puis 2 (pour P8), puis 4 (pour P4, P6 et P3).

Une demande utilisateur d'exécution d'un programme correspondra à une insertion (respectant le chaînage des valeurs de priorités croissantes). Une demande système de programme à exécuter correspondra à une suppression dans la liste du premier élément.

Exemple 10

FILE

ADR / PRIORITE	1	2	3	4	5	6	7	8	9	10	TETE	QUEUE
0		P2									2	2
1							P5	P1			7	8
2					P8						5	5
3											0	0
4			P4	P6	P3						3	5
5											0	0
6											0	0

REMARQUE

La première solution permet un gain d'espace mémoire, car il n'y a pas d'attribution de capacité fixe à chaque priorité, mais allocation d'une capacité globale.

La seconde solution permet un gain de temps d'exécution machine pour le programme d'insertion, car il n'y aura pas de défilement de liste.

■ Algorithme – exercice 18

Extraction d'un élément dans une file à niveau de priorité, représentée par une liste chaînée monodirectionnelle.

MÉTHODE

Nous voulons extraire le premier élément de la chaîne, car il est le plus prioritaire par définition.

- test sur l'existence d'un élément en attente
- sauvegarde de l'élément extrait
- mise à jour des pointeurs

```
procédure extraction-liste (données : info, priorité
   données modifiées : lien, début, debdisp
   résultat : élément)
début
   si début > 0
   alors élément ← info (début)
              (* mise à jour pointeur information *)
        svgd ← début
        début ← lien (début)
        (* mise à jour pointeurs disponibilités *)
```

```
            lien (svgd) ← debdisp
            debdisp ← svgd
      sinon afficher "file attente vide, extraction impossible"
      fsi
fin
```

■ Algorithme – exercice 19

Extraction d'un élément dans une file à niveau de priorité, représentée par un tableau de files.

MÉTHODE

Nous parcourons les différentes files dans l'ordre des numéros de priorité croissants. Nous extrayons le premier élément rencontré.
- traitement des files jusqu'à ce qu'un élément soit extrait (maxprio est le nombre maximum de priorité traitée)
- test sur l'existence d'un élément dans une file
- sauvegarde de l'élément extrait
- mise à jour des pointeurs tête et queue

```
procédure extraction-file
   (données : files, maxfile, maxprio
   données modifiées : tête, queue)
var trouvé : booléen
début
   trouvé ← .faux.
   prio ← 0
   répéter
     si tête (prio) > 0
     alors trouvé ← .vrai.
           élément ← files (prio, tête)
           si queue(prio) = tête(prio)
           alors queue(prio) ← 0
                 tête(prio) ← 0
           sinon tête(prio) ← tête (prio) + 1
                 si tête(prio) > maxfile
                 alors tête(prio) ← 1
                 fsi
           fsi
     fsi
     prio ← prio + 1
   jqa prio > maxprio ou trouvé
   si non trouvé
   alors afficher "file vide, extraction impossible"
   fsi
fin
```

■ Algorithme – exercice 20

Insertion d'un élément dans une file à niveau de priorité, représentée par une liste chaînée monodirectionnelle.

MÉTHODE

Nous insérons le nouvel élément, après tous les éléments de priorité égale, juste avant un élément de priorité strictement supérieure. L'algorithme demandé est un algorithme d'ajout après un élément donné. Attention, l'élément inséré peut être le premier du chaînage.

```
procédure insertion-liste
   (données modifiées : info, priorité, lien, début, debdisp)
var cpt, précédent : entiers
    sortie : booléen
début
   si debdisp = 0
   alors afficher "file attente complète, insertion impossible"
   sinon afficher "donner information et priorité à ajouter"
         saisir info (debdisp), priorité (debdisp)
               (* recherche place dans chaînage par priorité *)
         cpt ← début
         sortie ← .faux.
         tant que cpt > 0 et non sortie
               si priorité (debdisp) < priorité (cpt)
               alors sortie ← .vrai.
               sinon précédent ← cpt
                  cpt ← lien (cpt)
               fsi
         ftq
               (* mise à jour des pointeurs *)
         svgd ← debdisp
         debdisp ← lien (debdisp)
         si cpt = début
         alors lien (svgd) ← début
               début ← svgd
         sinon lien (svgd) ← cpt
                 lien (précédent) ← svgd
         fsi
   fsi
fin
```

■ Algorithme – exercice 18

Insertion d'un élément dans une file à niveau de priorité, représentée par un tableau de piles.

MÉTHODE

Cet algorithme est celui d'ajout d'un élément dans une file après sélection de la ligne du tableau traité.

- saisie de l'élément et de sa priorité
- recherche du numéro de case prévu pour l'ajout
- test de vérification de disponibilité
- insertion et mise à jour des pointeurs tête et queue

```
procédure insertion-file
   (données : maxfile, maxprio
   données modifiées : info, lien, début, debdisp)
```

```
var place, prio : entiers
    élément : caractères
début
   afficher "donner l'élément à insérer et sa priorité"
   saisir élément, prio
   si queue (prio) = maxfile
   alors place ← 1
   sinon place ← queue (prio) + 1
   fsi
   si place = tête (prio)
   alors afficher "file pleine pour cette priorité, ajout
                   impossible"
   sinon file (prio, place) ← élément
         queue (prio) ← place
         si tête (prio) = 0
         alors tête (prio) ← 1
         fsi
   fsi
fin
```

V – Aperçu des structures non linéaires

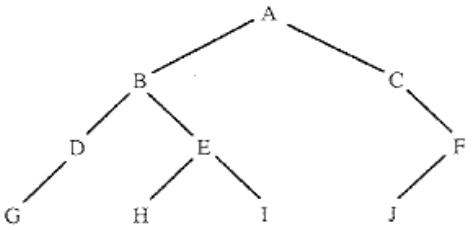
1 - ARBRES

Les structures de données non linéaires traduisent des relations hiérarchiques.

DÉFINITION

Un arbre binaire est une représentation où chaque élément appelé nœud possède 0, 1 ou 2 enfants que nous différencions comme enfant de gauche et enfant de droite.

Exemple 11



B enfant de gauche de A
C enfant de droite de A
D enfant de gauche de B
G enfant de gauche de D
pas d'enfant de droite pour D

Représentations

Deux représentations sont possibles :

– La représentation avec une liste chaînée utilise un tableau INFO, deux tableaux pointeurs DROITE et GAUCHE (contenant les adresses des enfants de droite et de gauche) et un pointeur RACINE (exemple 12).

– La représentation séquentielle utilise un tableau linéaire unique ayant les propriétés suivantes : la racine est en case 1, son enfant de gauche case 2, celui de droite case 3, et pour un élément case K, l'enfant de gauche est case (2 x K), celui de droite (2 x K) + 1 (exemple 13).

Exemple 12

La représentation de l'exemple 11 est :

ADR	INFO	GAUCHE	DROITE
1	C	0	5
2	J	0	0
3	A	7	1
4	I	0	0
5	F	2	0
6	E	8	4
7	B	9	6
8	H	0	0
9	D	10	0
10	G	0	0
11		12	
12		13	
.....	...	...	...

RACINE = 3

DEBDISP = 11

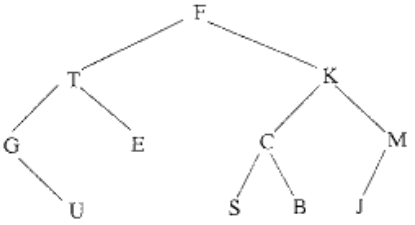
Exemple 13

La représentation de l'exemple 11 est :

ADR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
ARBRE	A	B	C	D	E		F	G		H	I			J			

❑ Exemple d'application 7

Soit l'arbre suivant :



1) Complétez sa représentation chaînée ?

ADR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
INFO	U		G	F	M		B	T	S	E	J	K		C	
GAUCHE		13				2							15		0
DROITE															

RACINE = ? DEBDISP

2) Quelle est sa représentation linéaire?

RÉPONSES

1)

ADR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
INFO	U		G	F	M		B	T	S	E	J	K		C	
GAUCHE	0	13	0	8	11	2	0	3	0	0	0	14	15	9	0
DROITE	0	.	1	12	0	.	0	10	0	0	0	5	.	7	

RACINE = 4 DEBDISP = 6

2)

ADR	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ARBRE	F	T	K	G	E	C	M		U			S	B	J	

■ Algorithme – exercice 22

DEFILEMENT des éléments d'un arbre. Les éléments s'affichent de la gauche vers la droite, en parcourant chaque branche jusqu'à son extrémité.

soit : F T G U E K C S B M J

a) Solution avec la liste

MÉTHODE

Nous traitons la branche de gauche de l'arbre jusqu'à son extrémité, tout en stockant dans une pile les adresses de tous les nœuds intermédiaires. A l'extrémité d'une branche, nous dépilons un élément et traitons alors son enfant de droite.

- Boucle de traitement de chaque nœud
- Boucle de traitement des enfants de gauche, avec sauvegarde par empilement
- Boucle de recherche d'un élément empilé qui possède un enfant de droite

```
procédure défile1  (données : info, gauche, droite, racine)
const maxpile c'est 100
var  pile(1:maxpile) : tableau d'entiers
     sommet, cpt : entiers
début
   cpt ← racine
   tant que cpt > 0
     tant que cpt > 0
         afficher info(cpt)
         push (cpt,maxpile,pile,sommet)
         cpt ← gauche(cpt)
     ftq
     répéter
         pop (pile,sommet,élément)
         cpt ← droite(élément)
     jqa cpt > 0 ou sommet = 0
   ftq
fin
```

b) Solution avec le tableau linéaire

MÉTHODE

Le défilement suit les mêmes règles que précédemment.

l'initialisation `cpt ← 1` remplace `cpt ← racine`

le calcul de l'élément de gauche

`cpt ← 2* cpt` remplace `cpt ← gauche(cpt)`

le calcul de l'élément de droite

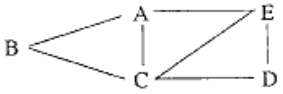
`cpt ← 2 * élément + 1` remplace `cpt ← droite(élément)`

2 - GRAPHES

DÉFINITION

Un graphe est constitué de deux types d'éléments : un ensemble d'éléments appelés NŒUDS et un ensemble d'ARETES identifiées par une paire unique de nœuds.

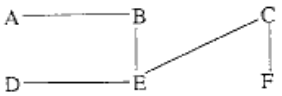
exemple 14 :



graphe

nœuds : A B C D E
arêtes : AB AC AE BC CD CE DE

exemple 15 :



graphe arborescent ou arbre

nœuds : A B C D E F
arêtes : AB BE CE CF DE

REMARQUE

Un arbre est un cas particulier de graphe tel qu'il n'existe qu'un chemin entre 2 nœuds. Le chemin est l'ensemble des arêtes reliant 2 nœuds donnés.

Exemple :
du nœud A vers le nœud E —> arêtes AB puis BE
du nœud D vers le nœud F —> arêtes DE, EC puis CF

Une matrice, ou tableau de dimension 2 est utilisée pour représenter un graphe.

Représentation de l'exemple 14 :

ADR	1	2	3	4	5
1	.	x	x		x
2	x	.	x		
3	x	x	.	x	x
4			x	.	x
5	x		x	x	.

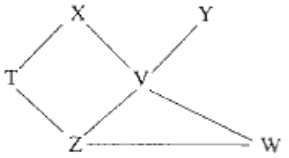
ADR	1	2	3	4	5
INFO	A	B	C	D	E

Représentation de l'exemple 15 :

ADR	1	2	3	4	5	6
1	.	x				
2	x	.			x	
3			.		x	x
4				.	x	
5		x	x	x	.	
6			x			.

ADR	1	2	3	4	5	6
INFO	A	B	C	D	E	F

❑ Exemple d'application 8



1) Quel est le tableau des nœuds qui ordonne alphabétiquement ses éléments ?
2) Quelle est la matrice associée à ce tableau ?

RÉPONSE

1)

ADR	1	2	3	4	5	6
INFO	T	V	W	X	Y	Z

2)

ADR	1	2	3	4	5	6
1	.			x		x
2		.	x	x	x	x
3		x	.			x
4	x	x		.		
5		x			.	
6	x	x	x			.

VI – Allocation dynamique

DÉFINITION

Les pointeurs que nous avons utilisés jusqu'à maintenant sont les indices des cases d'un tableau et chaque case possède une adresse physique d'implantation en mémoire. A l'appel d'un algorithme, le système affecte une adresse à chaque variable ou constante déclarées puis lors de l'exécution effectue constamment la conversion entre le nom de la variable ou constante utilisée et son adresse réelle. Une optimisation consiste à utiliser dans nos algorithmes directement ces adresses à la place des indices, ces adresses ne sont pas déclarées au début de l'algorithme, elles sont prêtées et rendues pendant le déroulement des instructions (allocation dynamique). Dans le traitement d'une liste, il n'y a plus de limitation dans le nombre d'informations chaînées, (seule la limite de la taille mémoire subsiste), et la gestion des disponibilités est faite par le système d'exploitation, dans le module gestion de la mémoire, et non par le programmeur. Lors de l'allocation d'une adresse mémoire (pointeur), le système sélectionne une valeur hexadécimale dans une zone mémoire appelée TAS.

Nouvelles instructions :

– Déclaration d'un pointeur :	var ptr : pointeur
– Initialisation d'un pointeur :	ptr ← NIL (Non Identified Link)
– Allocation d'une adresse mémoire disponible :	ALLOUE (ptr)
– Libération d'une adresse mémoire :	LIBERE (ptr)
– Utilisation d'une information pointée :	ptr • info
– Utilisation d'un lien pointé :	ptr • lien

REMARQUES :
– Le chaînage des éléments entre eux est à gérer par le programme.
– Il n'y a, en principe, pas de dépassement de capacité.
– La gestion des disponibilités n'est plus à la charge du programme.

■ Algorithme – exercice 22

CREATION, AJOUT, RECHERCHE, SUPPRESSION AVEC ALLOCATION DYNAMIQUE DE PLACE MEMOIRE

MÉTHODE
– Le programme de création ne comporte plus d'initialisation du chaînage des disponibilités car la liste des disponibilités est gérée par le système.
– Le programme d'insertion contient l'allocation d'une adresse mémoire et n'utilise plus la variable debdisp. Le test sur l'existence d'espace disponible disparaît.
– Le programme de suppression doit libérer la place mémoire utilisable pour un autre traitement.
– Lors des mises à jour de pointeurs, les différents programmes ne gèrent pas le chaînage des disponibilités.

```
procédure création (données : début, .info, .lien)
(* création d'une liste chaînée monodirectionnelle à en-tête
   circulaire. Seule l'initialisation de l'élément en-tête est
   nécessaire *)
var ptr : pointeur
début
   ALLOUE (ptr)
   début ← ptr
   ptr.info ← "en-tête"
   ptr.lien ← ptr
   ajout (début,.info,.lien)
fin


procédure ajout (donnée : début
                 données modifiées : .info, .lien)
var ptr,cpt,précédent : pointeurs
début
   ALLOUE (ptr)
   afficher "donner la nouvelle information"
```

```
   saisir ptr.info
   recherche (ptr.info,.info,.lien,début,cpt,précédent)
   ptr.lien ← cpt
   précédent.lien ← ptr
fin

procédure recherche   (données : élément,.info,.lien,début
                      résultats : cpt,précédent)
début
   précédent ← début
   cpt ← début.lien
   tant que cpt <> début et élément > cpt.info
     précédent ← cpt
     cpt ← cpt.lien
   ftq
fin


procédure suppression (donnée : début
                      données modifiées : .info,.lien)
var  cpt,précédent : pointeurs
     infosupp : caractères
début
   afficher "donner l'information à supprimer"
   saisir infosupp
   recherche (infosupp,.info,.lien,début,cpt,précédent)
   si cpt.info = infosupp
   alors  précédent.lien ← cpt.lien
          LIBERE(cpt)
   sinon afficher "information non trouvée, suppression impossible"
   fsi
fin
```

I – Introduction
II – Mise à jour d'un fichier d'organisation séquentielle
III – Edition sur imprimante
IV – Appareillage de deux fichiers
V – Mise à jour d'un fichier d'organisation séquentielle indexée
VI – Mise à jour d'un fichier d'organisation relative
VII – Application 1 : Gestion de bibliothèque
VIII – Application 2 : Gestion de stock
IX – Application 3 : Gestion de compte

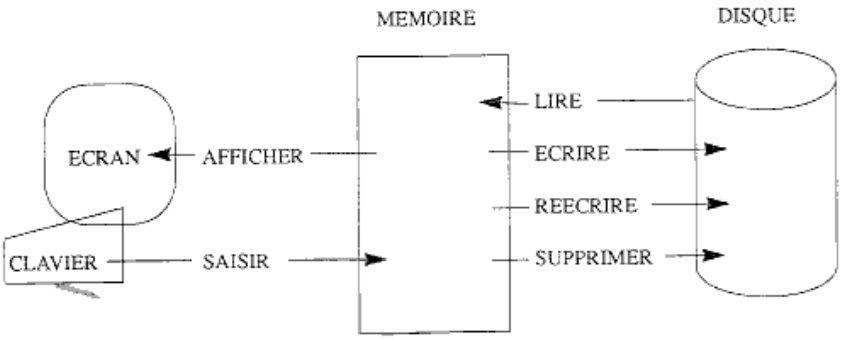
5

Les fichiers

I – Introduction

Toutes les structures de données que nous avons utilisées jusqu'à maintenant étaient stockées en mémoire, c'est-à-dire dans une zone volatile de la machine. En mémoire, la durée de vie d'une variable est égale au temps d'exécution du programme. Nous comprenons facilement que cette solution n'est pas envisageable pour l'informatisation d'une entreprise. Les données concernant un employé, un produit, un fournisseur ou un contrat doivent être mémorisées plus longtemps que la durée d'un programme. Les valeurs traitées doivent être recopiées sur un support non volatile, un disque, une disquette, une bande, une cassette ...

Nous connaissons les instructions pour échanger des valeurs entre l'utilisateur et la machine, à l'aide du clavier et de l'écran : SAISIR et AFFICHER, découvrons celles d'échanges entre l'ordinateur et le support non volatile d'informations : LIRE, ECRIRE, REECRIRE, SUPPRIMER, ...



Exemple 1

Supposons que nous avons stocké les noms et les années de naissance des employés d'une entreprise :

DUPONT | 58 | FONTAINE | 49 | DURAND | 61 | ABAR | 38 | ROLAND | 55 |

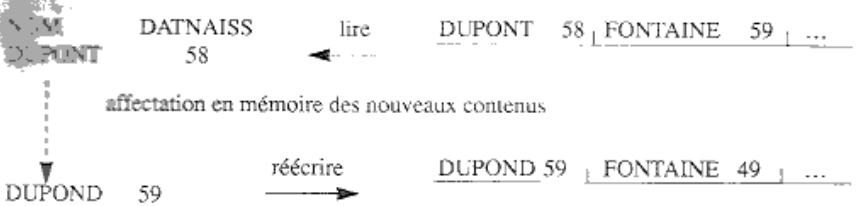
Une instruction de lecture, LIRE EMPLOYE, copie du disque vers la mémoire les informations d'UN employé. Deux variables déclarées en mémoire d'identifiants NOM et DATNAIS reçoivent les informations lues.



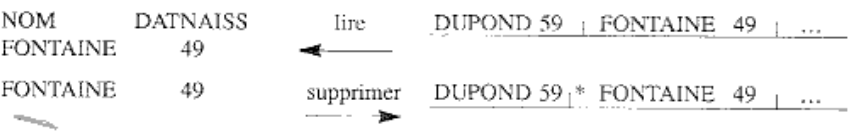
Une instruction d'écriture, ECRIRE EMPLOYE, copie les informations en mémoire sur une place libre du support :



Une instruction de réécriture, REECRIRE EMPLOYE, recopie les informations en mémoire à la place des dernières informations lues. Après l'instruction de lecture amenant les informations de l'employé "DUPONT" en mémoire, si nous ajoutons "1" à sa date de naissance et corrigeons l'orthographe du nom en remplaçant le "T" par un "D", nous obtenons :



Une instruction de suppression, SUPPRIMER EMPLOYE, met à jour un indicateur devant les dernières informations lues, pour indiquer que, logiquement, ces valeurs ne sont plus traitées. Nous parlons de suppression logique et non physique. La suppression physique nécessite d'effectuer un décalage de toutes les informations vers la gauche, mais ce traitement risque d'être très long ! Après la lecture des données concernant l'employé "FONTAINE", une instruction de suppression provoque ceci :



(le caractère "*" est supposé être l'indicateur de suppression)

DÉFINITION

Nous nommons ENREGISTREMENT (article ou enregistrement logique) la collection d'informations mémorisées pour définir UN des objets traités.

Nous appelons FICHIER la collection des enregistrements.

```
FICHIER    FIC-EMPLOYE
ENREGISTREMENT  rubriques :  nom-employé, date-de-naissance
                valeurs :    DUPOND,59
                             FONTAINE,49
                             DURAND ......
```

II – Mise à jour d'un fichier d'organisation séquentielle

DÉFINITION

Une première solution pour implanter les enregistrements sur le support est l'organisation séquentielle. Les enregistrements sont écrits les uns à la suite des autres, la lecture commence toujours par le premier, puis le second, son suivant et son suivant jusqu'au dernier.

DÉCLARATION ET INSTRUCTIONS

Nous déclarons un fichier ainsi :

```
FICHIER <nom-fichier> (ORGANISATION SEQUENTIELLE)
        ENREGISTREMENT <nom-enregistrement-fichier>
              <rubrique-1> : <type>
              <rubrique-2> : <type>
              <rubrique-3> : <type>
              ....
        FIN-ENREGISTREMENT
```

La première instruction du programme est l'ouverture du fichier, elle indique si le fichier va être lu (L) ou écrit (E), ou lu, écrit et mis à jour (L/E).

```
OUVRIR    <nom-fichier>    (L)
                           (E)
                           (L/E)
```

La dernière instruction du programme est la fermeture du fichier.

```
FERMER    <nom-fichier>
```

Les autres instructions ne concernent pas globalement le fichier mais un de ses enregistrement, nous les notons :

```
ECRIRE <nom-enregistrement-fichier>

LIRE <nom-enregistrement-fichier>
SI  FIN-FICHIER
ALORS <traitement-1>
SINON <traitement-2>
FSI
```

La lecture nécessite un test, car les lectures séquentielles se terminent lorsque tous les enregistrements ont été lus un à un. Un indicateur booléen du système de gestion de fichier nous permet d'identifier le fait qu'il n'y a plus d'enregistrement à lire, nous le nommons FIN-FICHIER.

```
REECRIRE nom-enregistrement-fichier

SUPPRIMER nom-enregistrement-fichier
```

Les deux dernières instructions ne peuvent être exécutées qu'après la lecture d'un enregistrement. C'est l'enregistrement le plus recemment lu qui est concerné par l'opération de réécriture ou de suppression. Seuls les supports adressables (disques et disquettes) permettent l'exécution de ces deux derniers ordres.

■ Algorithme – exercice 1

CREATION d'un fichier séquentiel sur disque des employés d'une entreprise, contenant le nom, le prénom, la date de naissance et la fonction de l'employé.

MÉTHODE

- Déclaration du fichier employé.
- Saisie des valeurs et stockage en mémoire.
- Validation des informations saisies.
- Ecriture des données de la mémoire sur le support.
- Question à l'utilisateur pour recommencer ou non le traitement de saisie.

```
programme création
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom        :  caractères
     e-prénom     :  caractères
     e-datnais    :  entier
     e-fonction   :  caractères
   fin-enregistrement
     (* notons la présence du préfixe e- pour différencier les rubriques
     des fichiers et les variables mémoires. La lettre "e" est simplement
     l'initiale du nom du fichier *)
var réponse, valid : caractères
```

```
début
    (* ouverture du fichier *)
   ouvrir employé (E)
   répéter
    afficher "donner le nom et le prénom"
    saisir e-nom, e-prénom
    afficher "donner la date de naissance"
    saisir e-datnais
    afficher "donner la fonction de l'employé"
    saisir e-fonction
    afficher "validez-vous les informations saisies (o/n) ?"
    saisir valid
    si  valid = "o"
    alors (* écriture sur disque *)
          écrire enr-employé
    fsi
        (* question à l'utilisateur *)
    afficher "autre employé à saisir (o/n) ?"
    saisir réponse
   jqa réponse = "n"
        (* fermeture du fichier *)
   fermer employé
fin
```

■ Algorithme – exercice 2

AFFICHAGE sur l'écran de toutes les informations contenues dans le fichier.

MÉTHODE

Nous prenons l'habitude d'effectuer une première lecture du fichier, pour différencier le cas d'un fichier vide (fin de fichier dès la première lecture), et le cas de la fin du traitement des enregistrements d'un fichier (fin de fichier rencontrée dans une boucle).

– Première lecture.

– Boucle de lecture séquentielle et d'affichage (nous faisons attention à traiter le premier enregistrement lu, avant d'effectuer une nouvelle lecture).

```
programme affiche
fichier employé (organisation séquentielle)
   enregistrement enr-employé
    e-nom       : caractères
    e-prénom    : caractères
    e-datnais   : entier
    e-fonction  : caractères
   fin-enregistrement
début
    (* ouverture du fichier *)
   ouvrir employé (L)
    (* première lecture *)
   lire enr-employé
   si fin-fichier
```

```
    alors afficher "le fichier des employés est vide"
    sinon afficher "affichage du fichier des employés"
          afficher " nom prénom date.naiss. fonction "
          répéter
                afficher e-nom, e-prénom, e-datnaiss, e-fonction
                lire enr-employé
          jqa fin-fichier
    fsi
    fermer employé
fin
```

■ Algorithme – exercice 3

CONSULTATION des informations stockées sur un employé de l'entreprise, nous le sélectionnons par son nom.

MÉTHODE
- Saisie du nom recherché.
- Lecture du premier enregistrement.
- Lecture du suivant jusqu'à ce que la recherche aboutisse ou échoue.
- Affichage des informations à consulter.

```
programme consultation
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin-enregistrement
var nom : caractères
   trouvé : booléen
début
    (* ouverture du fichier *)
   ouvrir employé (L)
   afficher "donner le nom recherché"
   saisir nom
    (* première lecture *)
   lire enr-employé
   si   fin-fichier
   alors afficher "le fichier des employés est vide"
   sinon (* recherche de l'enregistrement de l'employé *)
     recherche (employé, nom, trouvé)
               (* affichage du résultat de la recherche *)
     si non trouvé
     alors afficher nom, " n'est pas dans cette entreprise"
     sinon afficher e-nom, e-prénom, " né le ", e-datnaiss
           afficher "travaille dans cette entreprise "
           afficher "avec la fonction de ", e-fonction
     fsi
   fsi
    (* fermeture du fichier *)
   fermer employé
fin
```

```
procédure recherche    (données : employé, élément
                        résultat : trouvé)
var fini : booléen
début
   fini ← .faux.
   trouvé ← e-nom = élément
     (* recherche séquentielle *)
   tant que non trouvé et non fini
     lire enr-employé
     si fin-fichier
     alors fini ← .vrai.
     sinon trouvé ← e-nom = élément
     fsi
   ftq
fin
```

REMARQUE

Nous devons utiliser une boucle itérative et non répétitive, car le premier enregistrement déjà lu peut être celui recherché. Ceci nous oblige à utiliser un booléen (fini) pour repérer la fin de fichier, car le booléen du système (fin-fichier) ne peut être testé qu'une seule fois et juste après l'instruction de lecture. Nous utiliserons le booléen "fini" de nombreuses fois par la suite.

■ Algorithme – exercice 4

MODIFICATION de la fonction d'un employé, désigné par son nom.

MÉTHODE

– Saisie du nom de l'employé concerné par la modification.

– Recherche de l'enregistrement de cet employé dans le fichier (ce traitement est identique à celui effectué pour la consultation).

– Saisie de la nouvelle fonction.

– Mise à jour de l'enregistrement du disque.

```
programme modification
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin-enregistrement
var nom : caractères
   trouvé : booléen
début
   ouvrir employé (L/E)
   afficher "donner le nom de l'employé qui change de fonction "
   saisir nom
     (* recherche *)
   recherche (employé, nom, trouvé)
     (* traitement *)
   si   trouvé
```

```
    alors afficher "modification de ", e-nom, e-prénom
         afficher "fonction de ", e-fonction
         afficher "donner la nouvelle fonction "
         saisir e-fonction
              (* réécriture sur le disque *)
         réécrire enr-employé
    sinon afficher nom, " ne travaille pas dans cette entreprise !"
    fsi
    fermer employé
fin
```

■ Algorithme – exercice 5

SUPPRESSION d'un employé de l'entreprise.

MÉTHODE

Nous prenons l'habitude de demander une validation à l'utilisateur avant d'exécuter l'instruction de suppression logique dans le fichier. Un enregistrement peut contenir des informations très importantes et difficiles à reconstituer, et leur suppression par erreur n'est pas toujours réparable.

– Saisie du nom de l'employé qui quitte l'entreprise.

– Recherche de l'enregistrement concernant cet employé dans le fichier (ce traitement est identique à celui effectué pour la consultation ou la modification).

– Affichage de l'enregistrement et saisie de la validation de la suppression.

– Suppression logique de l'enregistrement.

```
programme suppression
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin-enregistrement
var nom, réponse : caractères
   trouvé : booléen
début
   ouvrir employé (L/E)
   afficher "donner le nom de l'employé qui part "
   saisir nom
     (* recherche *)
   recherche (employé, nom, trouvé)
     (* traitement *)
   si trouvé
   alors (* demande de validation *)
     afficher "validez-vous la suppression de ", e-nom, e-prénom
     afficher "né le ", e-datnaiss
     afficher "remplissant la fonction de ", e-fonction, " (O/N) ?"
     saisir réponse
```

```
        si  réponse = "O"
        alors     (* suppression sur disque *)
            supprimer enr-employé
        fsi
      sinon afficher nom, " ne travaille pas dans cette entreprise"
      fsi
      fermer employé
    fin
```

III – Edition sur imprimante

1 - ÉDITION SIMPLE

Nous différençons l'affichage à l'écran des informations d'un fichier et l'édition à l'imprimante de ces mêmes données. Nous constatons que chaque utilisateur possède toujours un écran individuel, mais rarement une imprimante personnelle, il ne peut pas imprimer quand il le désire. Nous effectuons des IMPRESSIONS DIFFEREES en constituant une copie de l'état désiré dans un fichier stocké sur le disque, et en demandant par la suite au système d'exploitation d'éditer ce fichier intermédiaire. L'imprimante n'est pas retardée par les calculs contenus dans le programme, ni par le temps de recherche des enregistrements, elle ne risque pas d'être mise hors service par un programme qui se termine de façon anormale.

Le fichier intermédiaire doit être déclaré, il est toujours organisé en séquentiel, ses enregistrements sont de la taille du papier d'impression (40, 80 ou 132 caractères). Nous différençons plusieurs types d'enregistrement : un ou plusieurs d'en-tête, un ou plusieurs de détail, un ou plusieurs de totaux. Nous numérotons séquentiellement chaque ligne distincte en la préfixant par le mot "ligne".

Exemple :

Etat d'édition des employés de la société. Nous remarquons l'importance de dater tous les listings.

en-tête :	ligne 1	ETAT DES EMPLOYES
	ligne 2	AU —/—/—
	ligne 3	NOM PRENOM DAT.NAISS. FONCTION
détail :	ligne 4	—— ——— —/—/— ———
		—— ——— —/—/— ———
		—— ——— —/—/— ———
		—— ——— —/—/— ———
total :	ligne 5	NOMBRE TOTAL D'EMPLOYES : ——

Nous déclarons chaque type de ligne, soit en constante, soit en variable. Dans le second cas, nous déclarons autant de variables que de rubriques non fixes à l'édition (nous prenons l'habitude de les préfixer par la lettre "l" et le numéro de ligne où elles se trouvent).

Cas de notre exemple :	ligne 1	constante	
	ligne 2	une variable :	
		date-du-jour	l2-date
	ligne 3	constante	
	ligne 4	quatre variables :	
		nom-employé	l4-nom
		prénom-employé	l4-prénom
		date-de-naissance	l4-date
		fonction	l4-fonction
	ligne 5	une variable :	
		total-employé	l5-tot

DÉCLARATION ET INSTRUCTION

Le fichier d'édition est déclaré comme suit :

```
FICHIER <nom-fichier-édition> (ORGANISATION SEQUENTIELLE)
    ENREGISTREMENT <nom-enregistrement-fichier-édition>
        <ligne> : caractères
    FIN-ENREGISTREMENT
```

L'instruction d'écriture dans ce fichier indique la ligne concernée par l'action d'ecriture.

```
ECRIRE <nom-enregistrement-fichier-édition> (ligne-x)
```

■ Algorithme – exercice 6

EDITION de toutes les informations stockées sur les employés de l'entreprise, de façon identique à l'exemple.

MÉTHODE

Nous nommons DATSYST la date système gérée par le système d'exploitation.

- Déclarer les fichiers employé et état.
- Déclarer les lignes.
- Première lecture du fichier employé.
- Ecriture des lignes d'en-tête.
- Initialisation du total.
- Boucle
  - écriture de la ligne détail.
  - incrément du total.
  - lecture de l'employé suivant.

– Ecriture de la ligne total.

```
programme édition
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom        :  caractères
     e-prénom     :  caractères
     e-datnais    :  entier
     e-fonction   :  caractères
   fin-enregistrement
fichier ETAT (organisation séquentielle)
   enregistrement enr-état
        ligne    :  caractères
   fin-enregistrement
const   ligne1, ligne3
var nbremploy : entier
   l2-date, l4-date, l5-tot : entiers
   l4-nom, l4-prénom, l4-fonction : caractères
début
   ouvrir employé (L)
          état (E)
   lire enr-employé
   si fin-fichier
   alors afficher "fichier employé vide, édition impossible"
   sinon écrire enr-état (ligne1)
        l2-date ← datsyst
        écrire enr-état (ligne2)
        écrire enr-état (ligne3)
        nbremploy ← 0
        répéter
             l4-nom ← e-nom
             l4-prénom ← e-prénom
             l4-date ← e-datnais
             l4-fonction ← e-fonction
             écrire enr-état (ligne4)
             nbremploy ← nbremploy + 1
                (* lecture du suivant *)
             lire enr-employ
        jqa fin-fichier
             (* dernière ligne d'édition *)
        l5-tot ← nbremploy
        écrire enr-état (ligne5)
   fsi
   fermer employé, état
fin
```

REMARQUE

Nous évitons d'utiliser la rubrique "l5-tot" comme variable de calcul dans le programme, nous utilisons une autre variable "nbremploy", car les rubriques d'un fichier d'impression ont couramment des formats d'édition et ne tolèrent pas d'être utilisées dans les traitements de calculs et de tests.

Exemples de format d'édition :

- Variable précédée de son signe "+" ou "–".
- Variable précédée des caractères "D" ou "C" (débit, crédit).
- Variable suivie des caractères "$" ou "Frs" (dollars, francs).
- Variable contenant une date et insérant le signe "/" entre le jour, le mois et l'année.

2 - ÉDITION AVEC GESTION DE PAGES

Les éditions nécessitent souvent une gestion des numéros de pages. Pour cela nous devons :

- déclarer les variables numéro de page et compteur de ligne ;
- incrémenter la variable compteur de ligne à chaque écriture dans le fichier d'édition;
- la comparer à une valeur indiquant le nombre maximum de lignes éditables sur les feuilles que nous possédons;
- effectuer un saut de page et l'édition de l'en-tête à chaque dépassement de capacité de page.

Les sauts de page ou de lignes sont gérés par le système de gestion de fichiers, un caractère particulier situé en début d'enregistrement indique la nécessité de positionner la tête d'écriture au début d'une nouvelle page ou de laisser quelques lignes blanches avant l'édition du prochain caractère.

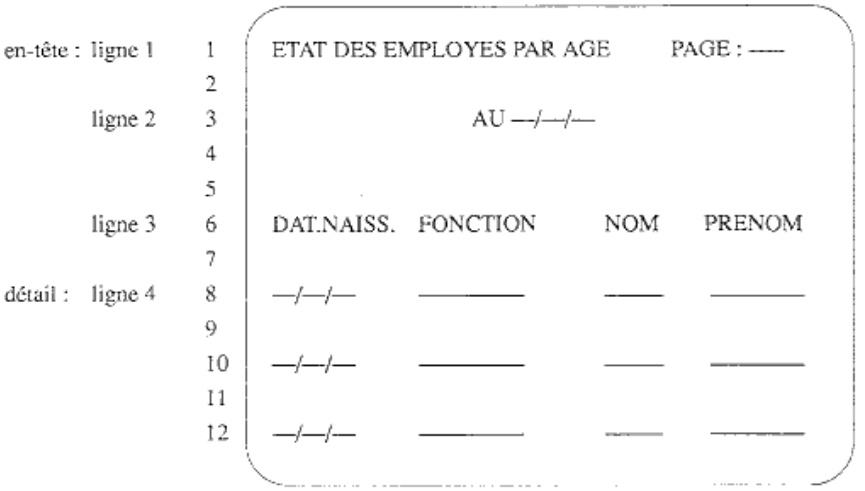
INSTRUCTIONS :

```
ECRIRE <nom-enr-fichier-édition> (ligne-x, après saut de page)

ECRIRE <nom-enr-fichier-édition> (ligne-x, après saut n lignes)
```

■ Algorithme – exercice 7

Edition du fichier des employés, trié par ordre décroissant sur la date de naissance. L'utilitaire de tri du système d'exploitation permet d'ordonner les enregistrements sur ce critère, et l'algorithme d'édition utilise le fichier résultat du tri. Nous possédons des feuilles de 50 lignes imprimables et désirons la présentation suivante.



MÉTHODE

- Déclaration des fichiers employé et état.
- Déclaration des lignes.
- Première lecture du fichier employé.
- Initialisation du compteur de page.
- Ecriture des lignes d'en-tête, incrément du compteur de page et initialisation du compteur de lignes.
- Boucle
  - comparaison du compteur de ligne et du maximum de lignes par page
  - en cas de dépassement, écriture des lignes d'en-tête incrément du compteur de page et réinitialisation du compteur de lignes
  - écriture de la ligne détail
  - incrément du compteur de lignes
  - lecture de l'employé suivant.

L'écriture des lignes d'en-tête apparaissant deux fois dans la méthode, les instructions correspondantes forment un sous-programme.

REMARQUE

Le test de dépassement de capacité de page est fait au début de la répétitive et non à la fin de celle-ci, pour éviter de préparer l'en-tête sur une nouvelle page, alors que nous venons de traiter le dernier enregistrement !

```
programme édition
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin-enregistrement
fichier état (organisation séquentielle)
    enregistrement enr-état
```

```
        ligne    : caractères
   fin-enregistrement
const   ligne3
var  l1-page,l2-date,l4-date,numpage,nbligne : entiers
   l4-nom,l4-prénom,l4-fonction : caractères
début
   ouvrir employé (L)
          état (E)
   lire enr-employé
   si   fin-fichier
   alors afficher "fichier employé vide, édition impossible"
   sinon numpage ← 0
         l2-date ← datsyst
         en-tête (ligne2, ligne3, ligne1, état, numpage, nbligne)
         répéter
             si   nbligne >= 50
             alors entête (ligne2, ligne3, ligne1, état, numpage, nbligne)
             fsi
             l4-nom ← e-nom
             l4-prénom ← e-prénom
             l4-date ← e-datnais
             l4-fonction ← e-fonction
             écrire enr-état (ligne4, après saut 1 ligne)
             nbligne ← nbligne + 2
             lire enr-employé
         jqa fin-fichier
   fsi
   fermer employé, état
fin

procédure entête
   (données : ligne2, ligne3
   données modifiées : ligne1, état, numpage
   résultat : nbligne)
début
   numpage ← numpage + 1
   l1-page ← numpage
   écrire enr-état (ligne1, après saut de page)
   écrire enr-état (ligne2, après saut 1 ligne)
   écrire enr-état (ligne3, après saut 2 lignes)
   nbligne ← 6
fin
```

3 - ÉDITION AVEC RUPTURES

■ Algorithme – exercice 8

Edition des employés regroupés par fonctions. L'algorithme est écrit en supposant que le tri des enregistrements a été effectué précédemment par un utilitaire du système d'exploitation, seule la mise en page est à prévoir.

en-tête	ligne 1	ETAT DES EMPLOYES PAR FONCTION
	ligne 2	AU —/—/—
en-tête-fonction	ligne 3	FONCTION : ————
	ligne 4	NOM PRENOM DATE DE NAISSANCE
détail	ligne 5	——— ——— —/—/—
		——— ——— —/—/—
		——— ——— —/—/—
total-fonction	ligne 6	NOMBRE EMPLOYES : ——
		•••••••••••••••••
		de nouveau des lignes 3 à 6
		•••••••••••••••••
total	ligne 7	NOMBRE TOTAL D'EMPLOYES : ——

MÉTHODE

- Déclaration des fichiers employé et état.
- Déclaration des lignes d'édition.
- Première lecture du fichier employé.
- Ecriture des lignes d'en-tête.
  Initialisation du total-entreprise.
- Boucle – écriture des lignes d'en-tête-fonction
  initialisation du total-fonction
  – boucle – écriture de la ligne détail
  incrément du total-fonction
  – lecture de l'employé suivant
  – écriture de la ligne total-fonction
  ajout du total-fonction au total-entreprise
- Ecriture de la ligne total-entreprise.

REMARQUE

La difficulté est de trouver les conditions d'arrêt des deux boucles utilisées :
– La première attend la fin de fichier pour se terminer, elle utilise un booléen du programme FINI et non le booléen du système FIN-FICHIER, car le test d'arrêt ne se trouve pas immédiatement après la lecture séquentielle du fichier. (Rappel : le test du booléen du système n'est réalisable qu'une seule fois et immédiatement après l'instruction de lecture !)
– La seconde boucle attend, soit le changement de fonction du prochain employé à traiter, soit la fin du fichier. Le changement de fonction, appelé rupture, est difficile à détecter, car il n'y a en mémoire, à un moment donné, qu'un enregistrement. Il est donc impossible de comparer le contenu de la rubrique "fonction" actuellement en mémoire avec le contenu qui s'y trouvait précédemment ! Pour toute rupture, nous devons stocker le contenu de la variable servant à la déceler dans une case mémoire particulière, nous l'appelons "nom-variable-TRAITEE".

```
programme édition-rupture
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin-enregistrement
fichier état (organisation séquentielle)
   enregistrement enr-état
        ligne :  caractères
   fin-enregistrement
const   ligne1, ligne4
var  l2-date, l5-date, l6-tot, l7-tot : entiers
     l3-fonction, l5-nom, l5-prénom : caractères
     tot-fonction, tot-entreprise : entiers
     fonction-traitée : caractères
     fini : booléen
début
   ouvrir employé (L)
          état (E)
   lire enr-employé
   si fin-fichier
   alors afficher "fichier employé vide, édition impossible "
   sinon fini ← .faux.
         l2-date ← datsyst
              (* contraction de deux instructions d'écriture édition *)
         écrire enr-état (ligne1, ligne2)
         tot-entreprise ← 0
         répéter
               l3-fonction ← e-fonction
               écrire enr-état (ligne3, ligne4)
               fonction-traitée ← e-fonction
               tot-fonction ← 0
               répéter
                  l5-nom ← e-nom
                  l5-prénom ← e-prénom
                  l5-date ← e-datnais
                  écrire enr-état (ligne5)
                  tot-fonction ← tot-fonction + 1
                     (* lecture suivant *)
                  lire enr-employé
                  si fin-fichier
                  alors fini ← .vrai.
                  fsi
               jqa fini ou fonction-traitée <> e-fonction
               l6-tot ← tot-fonction
               écrire enr-état (ligne6)
               tot-entreprise ← tot-entreprise + tot-fonction
         jqa fini
         l7-tot ← tot-entreprise
         écrire enr-état (ligne7)
   fsi
   fermer employé, état
fin
```

■ Algorithme – exercice 9

Nous voulons obtenir l'état suivant, à partir du fichier RESULTAT contenant un article par bureau de vote et trié sur département, ville et bureau.

Fichier RESULTAT organisation séquentielle

- numéro de département r-dep
- nom de ville r-vil
- numéro de bureau de vote r-bur
- nombre d'inscrits r-ins
- nombre de votants r-vot
- nombre de suffrages par liste r-list (1:3) tableau

```
1                          RESULTAT DE VOTE
2
3                          DU   —/—/—
4
5
6          DEPARTEMENT  :    ——
7
8                VILLE        :   ————————————
9
10  BUREAU  INSCRITS  VOTANTS  EXPRIMES  LISTE1  LISTE2  LISTE3
11  ——      ———      ——       ———       ———     ———     ———
12  ——      ———      ———      ———       ———     ———     ———
13  ——      ———      ———      ———       ———     ———     ———
14
15    VILLE (EN %)     :   —— %                  —%      —%      —%
16
17    reprise des lignes 8 à 15 pour les autres villes du même département...
18
19  DEPARTEMENT (EN %) :  —— %                  —%      —%      —%
20
21  ...reprise des lignes 6 à 19 pour les autres départements
22  ...
23  PAYS :    INSCRITS  VOTANTS  EXPRIMES  LISTE1  LISTE2  LISTE3
24
25  TOTAUX    ———      ———      ———       ———     ———     ———
26
27  EN POURCENTAGE     —%                  —%      —%      —%
```

NB : nous affichons les pourcentages en valeurs entières.

TRAVAIL À FAIRE :

1) Numéroter les lignes d'édition (ligne1, ligne2, etc.).
2) Regrouper les lignes d'en-tête, de détails et de totaux.
3) Définir les variables nécessaires pour les totaux intermédiaires.
4) Ecrire le programme principal de l'édition (chaque traitement d'en-tête, de détail ou de total formera un sous-programme).
5) Ecrire les différents sous-programmes.
6) Simplifier l'écriture des sous-programmes, en regroupant les instructions communes à plusieurs d'entre eux.

RÉPONSES

1) Associons les numéros écrits en marge gauche de l'état et les différents noms de lignes (voir ci-dessous).
2) Trois blocs d'en-tête, un bloc de détail et trois blocs de totaux existent.
3) Le pourcentage de votants est le total du nombre de votants, divisé par le nombre total d'inscrits (voir ci-dessous).

Le pourcentage de suffrages pour chacune des trois listes est égal au total des suffrages de la liste, divisé par le total des suffrages exprimés (total des bulletins pour les trois listes et non total des votants qui contient la somme des bulletins blancs ou nuls).

Pour tous les calculs d'un niveau de rupture, cinq variables de stockage suffisent !

	1)		2)		3)
n°	1	ligne1	)	en-tête pays	
n°	3	ligne2	)		
n°	6	ligne3		en-tête département	
n°	8	ligne4	)	en-tête ville	
n°	10	ligne5	)		
n°	11	ligne6		détail-bureau	
n°	15	ligne7		total-ville	vil-ins, vil-vot, vil-list(3)
n°	19	ligne8		total-département	dep-ins, dep-vot, dep-list(3)
n°	23	ligne9	)		
n°	25	ligne10	)	total-pays	pay-ins, pay-vot, pay-list(3)
n°	27	ligne11	)		

4) MÉTHODE

– Déclaration des fichiers résultat et état.
– Déclaration des lignes d'édition.
– Première lecture du fichier résultat.
– 3 boucles imbriquées gérant les trois blocs d'en-têtes, le bloc détail contenant la lecture du fichier résultat et les trois blocs de totaux.

```
programme édition-rupture
fichier résultat (organisation séquentielle)
    enregistrement enr-résultat
        r-dep :  entier
```

```
            r-vil :  caractères
            r-bur :  entier
            r-ins :  entier
            r-vot :  entier
            r-list (1:3)  :      tableau d'entiers
        fin-enregistrement
    fichier état (organisation séquentielle)
        enregistrement enr-état
            ligne :  caractères
        fin-enregistrement
    const ligne1, ligne5, ligne9
    var  (* édition *)
       l2-date,l3-dep,l6-bur,l6-ins,l6-vot,l6-exp : entiers
       l4-vil : caractères
       l7-%vot,l8-%vot,l10-ins,l10-exp,l10-vot,l11-%vot : entiers
       l6-list(1:3),l7-%list(1:3),l8-%list(1:3) : tableau d'entiers
       l10-list(1:3),l11-%list(1:3) : tableau d'entiers
         (* cumuls *)
       vil-ins,dep-ins,pay-ins,vil-vot,dep-vot,pay-vot : entiers
       vil-list(1:3),dep-list(1:3),pay-list(1:3) : tableau d'entiers
         (* rupture *)
       vil-traité,dep-traité : entiers
       fini : booléen
    début
       ouvrir résultat (L)
              état (E)
       lire enr-résultat
       si fin-fichier
       alors afficher "fichier résultat vide, édition impossible "
       sinon fini ← .faux.
             en-tête-pays (ligne1,ligne2,état,pay-ins,pay-vot,pay-list)
             répéter
                   en-tête-département (résultat,ligne3,état,dep-
                   ins,dep-vot,dep-list,dep-traité)
                   repéter
                      en-tête-ville (resultat,ligne5,ligne 4,
                      état, vil-ins,vil-vot,vil-list,vil-traité
                      répéter
                         détail-bureau (résultat,ligne6,état,vil-
                                        ins,vil-vot,vil-list)
                         lire enr-résultat
                         si fin-fichier
                         alors      fini ← .vrai.
                         fsi
                      jqa fini ou vil-traité <> r-vil
                      total-ville (vil-ins,vil-vot,vil-list,ligne7,
                                   état,dep-ins,dep-vot,dep-list)
                   jqa fini ou dep-traité <> r-dep
                   total-département (dep-ins,dep-vot,dep-list, ligne8,
                                      état,pay-ins,pay-vot,pay-list)
             jqa fini
```

```
        total pays (pay-ins,pay-vot,pay-list,ligne9,ligne10,ligne11,état)
      fsi
      fermer résultat, état
   fin
```

REMARQUE

Dans le cas où deux enregistrements consécutifs ont le même nom de ville, bien que leurs numéros de département soient différents, la rupture sur département n'est pas détectée. Pour conserver à l'algorithme une certaine simplicité, nous avons éliminé ce cas peu probable, vu le nombre de villes traitées et le tri par ordre alphabétique. Pour le gérer, nous devons changer le test d'arrêt de la répétitive interne à la structure :

```
répéter
   répéter
     répéter
        ....
     jqa fini ou vil-traité <> r-vil OU DEP-TRAITE <> R-DEP
   jqa fini ou dep-traité <> r-dep
jqa fini
```

5) MÉTHODE

– Un bloc en-tête à trois fonctions :
- initialisation des totaux
- écriture des lignes d'en-tête
- sauvegarde de la variable de rupture

– Un bloc de total à deux fonctions :
- écriture des lignes de totaux
- cumuls pour calculer les totaux de la rupture suivante

– Le bloc détail est identique à un bloc de total, sa particularité est d'être suivi par la lecture du fichier.

```
procédure en-tête-pays
   (données : ligne1
   données modifiées : ligne2,état
   résultat : pay-ins,pay-vot,pay-list)
var cpt : entier
début
     (* initialisation *)
   pay-ins ← 0
   pay-vot ← 0
   pour  cpt ← 1 jqa 3
         pay-list(cpt) ← 0
   fpour
     (* écriture *)
   l2-date ← datsyst
   écrire enr-état (ligne1, après saut de page)
   écrire enr-état (ligne2, après saut 1 ligne)
fin
```

```
procédure en-tête-département
   (données : résultat
   données modifiées : ligne3,état
   résultat : dep-ins,dep-vot,dep-list,dep-traité)
var cpt : entier
début
     (* initialisation *)
   dep-ins ← 0
   dep-vot ← 0
   pour cpt ← 1 jqa 3
         dep-list(cpt) ← 0
   fpour
     (* écriture *)
   l3-dep ← r-dep
   écrire enr-état (ligne3, après saut 2 lignes)
     (* sauvegarde *)
   dep-traité ← r-dep
fin


procédure en-tête-ville
   (données : résultat,ligne5
   données modifiées : ligne4,état
   résultat : vil-ins,vil-vot,vil-list,vil-traité)
var cpt : entier
début
     (* initialisation *)
   vil-ins ← 0
   vil-vot ← 0
   pour cpt ← 1 jqa 3
     vil-list(cpt) ← 0
   fpour
     (* écriture *)
   l4-vil ← r-vil
   écrire enr-état (ligne4, après saut 1 ligne)
   écrire enr-état (ligne5, après saut 1 ligne)
     (* sauvegarde *)
   vil-traité ← r-vil
fin


procédure détail-bureau
   (données : résultat
   données modifiées : ligne6,état,vil-ins,vil-vot,vil-list)
var nbexp,cpt : entiers
début
     (* écriture *)
   l6-bur ← r-bur
   l6-ins ← r-ins
   nbexp ← 0
   pour cpt ← 1 jqa 3
         l6-list(cpt) ← r-list(cpt)
```

```
        nbexp ← nbexp + r-list(cpt)
                (* cumul tableau ville *)
        vil-list(cpt) ← vil-list(cpt) + r-list(cpt)
    fpour
    16-vot ← r-vot
    16-exp ← nbexp
    écrire enr-état (ligne6)
      (* cumul inscrits et votants ville *)
    vil-ins ← vil-ins + r-ins
    vil-vot ← vil-vot + r-vot
fin


procédure total-ville
   (données : vil-ins,vil-vot,vil-list
   données modifiées : ligne7,état,dep-ins,dep-vot,dep-list)
var nbexp,cpt : entiers
début
     (* écriture *)
   nbexp ← vil-list(1) + vil-list(2) + vil-list(3)
   17-%vot ← vil-vot / vil-ins
   pour cpt ← 1 jqa 3
        17-%list(cpt) ← vil-list(cpt) / nbexp
                (* cumul tableau département *)
        dep-list(cpt) ← dep-list(cpt) + vil-list(cpt)
   fpour
   écrire enr-état (ligne7, après saut 1 ligne)
     (* cumul inscrits et votants département *)
   dep-ins ← dep-ins + vil-ins
   dep-vot ← dep-vot + vil-vot
fin


procédure total-département
   (données : dep-ins,dep-vot,dep-list
   données modifiées : ligne8,état,pay-ins,pay-vot,pay-list)
var nbexp, cpt : entiers
début
   nbexp ← dep-list(1) + dep-list(2) + dep-list(3)
   18-%vot ← dep-vot / dep-ins
   pour cpt ← 1 jqa 3
        18-%list(cpt) ← dep-list(cpt) / nbexp
                (* cumul tableau pays *)
        pay-list(cpt) ← pay-list(cpt) + dep-list(cpt)
   fpour
   écrire enr-état (ligne8, après saut 1 ligne)
        (* cumul inscrits et votants pays *)
   pay-ins ← pay-ins + dep-ins
   pay-vot ← pay-vot + dep-vot
fin
```

```
procédure total-pays (données : pay-ins,pay-vot,pay-list,ligne9
                      données modifiées : ligne10,ligne11,état)
var nbexp,cpt : entiers
début
    (* écriture *)
   nbexp ← pay-list(1) + pay-list(2) + pay-list(3)
   l10-ins ← pay-ins
   l10-vot ← pay-vot
   l10-exp ← nbexp
   l11-%vot ← pay-vot / pay-ins
   pour cpt ← 1 jqa 3
        l10-list(cpt) ← pay-list(cpt)
        l11-%list(cpt) ← pay-list(cpt) / nbexp
   fpour
   écrire enr-état (ligne9, après saut 1 ligne)
   écrire enr-état (ligne10, après saut 1 ligne)
   écrire enr-état (ligne11, après saut 1 ligne)
fin
```

6) SIMPLIFICATION

Les trois sous-programmes des en-têtes ou des totaux sont si proches que nous pouvons réduire le nombre de lignes de l'algorithme.

Les instructions communes aux trois sous-programmes "en-tête" sont les suivantes :

```
procédure en-tête-param
(données : lignex
données modifiées : état
résultat : rup-ins,rup-vot,rup-list)
var cpt : entier
début
    (* initialisation *)
   rup-ins ← 0
   rup-vot ← 0
   pour cpt ← 1 jqa 3
     rup-list(cpt) ← 0
   fpour
     (* écriture *)
   écrire enr-état (lignex)
fin
```

Définissons maintenant les instructions spécifiques à chacun des sous-programmes "en-tête" :

```
procédure en-tête-pays
   (données : ligne1
   données modifiées : ligne2,état
   résultat : pay-ins,pay-vot,pay-list)
début
   l2-date ← datsyst
```

```
      écrire enr-état (ligne1)
      en-tête-param (ligne2,état,pay-ins,pay-vot,pay-list)
   fin

   procédure en-tête-département
      (données : résultat
      données modifiées : ligne3,état
      résultat : dep-ins,dep-vot,dep-list,dep-traité)
   début
      l3-dep ← r-dep
      dep-traité ← r-dep
      en-tête-param (ligne3,état,dep-ins,dep-vot,dep-list)
   fin

   procédure en-tête-ville
      (données : résultat,ligne5
      données modifiées : ligne4,état
      résultat : vil-ins,vil-vot,vil-list,vil-traité)
   début
      l4-vil ← r-vil
      écrire enr-état (ligne4)
      vil-traité ← r-vil
      en-tête-param (ligne5,état,vil-ins,vil-vot,vil-list)
   fin
```

Les instructions communes aux sous-programmes "total-ville" et "total-département" sont les suivantes :

```
   procédure total-param
      (données : rup-ins,rup-vot,rup-list
      données modifiées : lignex,état,rup2-ins,rup2-vot,rup2-list)
   var  nbexp,cpt : entiers
   début
      nbexp ← rup-list(1) + rup-list(2) + rup-list(3)
      lx-%vot ← rup-vot / rup-ins
      pour  cpt ← 1 jqa 3
            lx-%list(cpt) ← rup-list(cpt) / nbexp
                  (* cumul tableau pour préparer rupture suivante *)
            rup2-list(cpt) ← rup2-list(cpt) + rup-list(cpt)
      fpour
      écrire enr-état (lignex)
        (* cumul inscrits et votants pour rupture suivante *)
      rup2-ins ← rup2-ins + rup-ins
      rup2-vot ← rup2-vot + rup-vot
   fin
```

Il n'existe aucune instruction spécifique à chacun des sous-programmes "total-ville" et "total-département" (leurs appels sont remplacés par un appel de "total-param"). Les procédures "détail-bureau" et "total-pays" ne se prêtent pas au regroupement, car leurs lignes d'édition sont particulières.

IV – Appareillage de deux fichiers

■ Algorithme – exercice 10

Soient les fichiers suivants :

produit	org : seq	support : disque
numéro de produit	p-num	
désignation	p-design	
quantité en stock	p-stock	
prix unitaire	p-prix	
commande-du-jour	org : seq.	
numéro de client	cj-numcli	
numéro de produit	cj-numpro	
quantité commandée	cj-qtecom	
commande-en-attente	org : seq	

idem au fichier commande-du-jour (ca-numcli, ca-numpro, ca-qtecom)

commande-livrable org : seq

idem au fichier commande-du-jour (cl-numcli, cl-numpro, cl-qtecom)

CREATION des fichiers commande-en-attente et commande-livrable, à partir des fichiers produit et commande-du-jour, sachant qu'une commande n'est jamais livrée partiellement si la quantité en stock du produit est insuffisante. Les fichiers commande-du-jour et produit sont traités après avoir été TRIES tous les deux par numéro de produit.

Expliquons le déroulement de l'algorithme à partir d'un exemple.

PRODUIT				COMMANDE-DU-JOUR		
n°	**désign**	**stock**	**prix**	**n° cli.**	**n° pro.**	**qte**
001	TABLE	010	0320.00	1764	001	1
003	CHAISE	025	0078.50	6732	004	2
004	TABOURET	003	0045.00	1234	004	4
007	FAUTEUIL	000	0117.75	8888	006	2
013	LAMPE	100	0250.00	1234	007	3

Un produit peut ne pas être commandé, ex. : le 003.

Un produit peut être commandé plusieurs fois, ex. : le 004.

Un numéro de produit inexistant peut être commandé par erreur, ex. : le 006.

Nous détectons tous les cas possibles par la comparaison des numéros de produit des deux fichiers, trois cas se réalisent : < , > , = .

n°	produit	n° prod. commandé	comparateur	actions
->	001	-> 001	=	– 1 au stock
				lecture commande suivante
	001	-> 004	<	lecture produit suivant
->	003	004	<	lecture produit suivant
->	004	004	=	– 2 au stock
				lecture commande suivante
	004	-> 004	=	"stock insuffisant"
				lecture commande suivante
	004	-> 006	<	lecture produit suivant
->	007	006	>	"produit 006 inexistant"
				lecture commande suivante
	007	-> 007	=	"stock insuffisant"
				lecture commande suivante
	007	-> fin	?	arrêt du programme

TROIS DIFFICULTÉS :

– Ce traitement ne nécessitant pas la présence d'un utilisateur, qui va lire nos messages d'erreur ? (Solution : les enregistrements provoquant une erreur sont écrits dans un fichier spécial nommé ERREUR.)

– Nous ne pouvons effectuer qu'une réécriture sur le fichier produit pour une lecture, nous devons terminer la mise à jour de la variable p-stock avant d'effectuer l'instruction de réécriture !

– Que faire si le fichier des produits se termine avant celui des commandes (ex. : un produit n° 016 a été commandé par erreur) ?

Solution 1 : petit programme en fin de traitement pour gérer ce cas particulier.

Solution 2 : modifier le contenu de la variable p-num pour que le programme se poursuive sans aucune lecture supplémentaire sur le fichier produit, pour cela p-num doit être toujours supérieur à c-numpro (nous lui donnons la valeur HV, Haute Valeur ou High Value).

MÉTHODE

- Définition des cinq fichiers produit, comm-jour, comm-att, comm-liv, erreur.
- Première lecture du fichier comm-jour.
- Première lecture du fichier produit.
- Boucle – comparaison des numéros de produit
  - cas p-num < cj-numpro (produit p-num non commandé)
    - lecture du produit suivant
  - cas p-num > cj-numpro (commande d'un produit inexistant)
    - écriture dans le fichier erreur
    - lecture de la commande suivante
  - cas p-num = c-numpro (produit commandé trouvé)

- boucle de traitement de toutes les commandes de ce produit
  - décrément du stock, ou message insuffisant
  - lecture de la commande suivante
- réécriture du produit mis à jour
- lecture du produit suivant

REMARQUE

Ce programme s'arrête dès que le fichier des commandes est lu dans sa totalité.

```
programme appareillage
fichier produit (organisation séquentielle)
   enregistrement enr-produit
     (* détail dans le texte de l'exercice *)
fichier comm-jour (organisation séquentielle)
   enregistrement enr-comjour
fichier comm-att (organisation séquentielle)
     enregistrement enr-comatt
fichier comm-liv (organisation séquentielle)
   enregistrement enr-comliv
fichier erreur (organisation séquentielle)
   enregistrement enr-erreur
     e-numcli :  entier
     e-numpro :  entier
     e-qtecom :  entier
   fin-enregistrement
var fin-commande, problème : booléen
début
   ouvrir comm-jour (L)
          produit (L/E)
          comm-att, comm-liv, erreur (E)
   problème ← .faux.
   lire enr-comjour
   si fin-fichier
   alors afficher "fichier commande du jour vide, pas de traitement"
        problème ← .vrai.
   fsi
   lire enr-produit
   si fin-fichier
   alors afficher "fichier produit vide, pas de traitement"
        problème ← .vrai.
   fsi
   si non problème
   alors  (* boucle *)
        fin-commande ← .faux.
        répéter
                (* comparaison *)
             si p-num < cj-numpro
             alors    (* produit non commandé *)
                  lireprod   (produit)
```

```
            sinon si p-num > cj-numpro
              alors  (* produit commandé inexistant *)
                     (* ce test est toujours vrai lorsque le
fichier                   produit a été totalement lu *)
                  enr-erreur ← enr-comjour
                  écrire erreur
                  lirecomm (commande, fin-commande)
               sinon  (* traitement de la commande *)
                  trait-comm (comm-jour,produit,fin-commande,
                  comm-att,comm-liv)
                  réécrire produit
                  lireprod  (produit)
              fsi
            fsi
        jqa fin-commande
       fsi
     fin



     procédure lirecomm
        (données : commande
        résultat : fin-commande)
     début
        lire enr-comjour
        si fin-fichier
        alors fin-commande ← .vrai.
        fsi
     fin


     procédure lireprod (données modifiées : produit)
     début
        lire produit
        si fin-fichier
        alors p-num ← HV
        fsi
     fin


     procédure trait-comm  (données : comm-jour
                  données modifiées : produit
                  résultats : fin-commande, comm-att, comm-liv)
     var autre-produit : booléen
     début
        répéter
          si cj-qtecom  <= p-stock
          alors     p-stock ← p-stock - cj-qtecom
              enr-comliv ← enr-comjour
              écrire enr-comliv
          sinon     enr-comatt ← enr-comjour
```

```
            écrire enr-comatt
        fsi
        lirecomm (commande, fin-commande)
        si  fin-commande
        alors     autre-produit ← .vrai.
        sinon     autre-produit ← p-num = cj-numpro
        fsi
      jqa autre-produit
fin
```

■ Algorithme – exercice 11

Soit le fichier suivant, pour effectuer la mise à jour du fichier des employés déjà traité.

mise-à-jour org : seq

code màj	m-màj	C pour création M pour modification S pour suppression
nom employé	m-nom	
prénom	m-prénom	valeur existe pour création
date de naissance	m-datnais	valeur existe pour création
fonction	m-fonction	valeur existe pour création et modification

Le fichier "employé" est préalablement trié sur le nom des employés (nous supposons qu'il n'y a pas d'homonymes dans la société). Le fichier "mise-à-jour" est trié sur le nom de l'employé et le code de mise à jour, car un enregistrement peut être créé puis modifié, puis supprimé, dans le même traitement. La création d'un enregistrement supplémentaire dans un fichier séquentiel s'effectuant à la fin du fichier, le critère de tri ne sera plus respecté et la suite de l'appareillage impossible, nous allons donc créer un nouveau fichier des employés mis à jour.

MÉTHODE

- Déclaration de quatre fichiers : employé, mise-à-jour, employé-maj et erreur.
- Premières lectures des deux fichiers à appareiller.
- Boucle – comparaison des noms du fichier employé et du fichier mise-à-jour
  - cas > et code "C" fichier mise-à-jour pris en compte
    traitement des modifications ou suppression éventuelles de ce nouvel employé
    et code "M" ou "S" message d'erreur
  - cas < employé non concerné par la mise à jour
    recopie sans modification dans le fichier employé-maj
  - cas = et code "M" prise en compte de la nouvelle fonction
    et code "S" la suppression équivaut à la non-écriture dans le fichier employé-maj
    et code "C" message d'erreur

```
programme maj-employé
fichier employé (organisation séquentielle)
   enregistrement enr-employé
     e-nom       : caractères
     e-prénom    : caractères
     e-datnais   : entier
     e-fonction  : caractères
   fin enregistrement
fichier mise-à-jour (organisation séquentielle)
   enregistrement enr-maj
     m-maj : caractères
     m-zones-communes
         m-nom      : caractères
         m-prénom   : caractères
         m-datnais  : entier
         m-fonction : caractères
   fin-enregistrement
fichier employé-maj (organisation séquentielle)
   enregistrement enr-empmaj
     em-nom      : caractères
     em-prénom   : caractères
     em-datnais  : entier
     em-fonction : caractères
   fin-enregistrement
fichier erreur (organisation séquentielle)
   enregistrement enr-erreur
     er-nom    : caractères
     er-maj    : caractères
   fin-enregistrement
début
   ouvrir mise-à-jour, employé (L)
          erreur, employé-maj (E)
   liremaj (mise-à-jour)
   lireemp (employé)
   si m-nom = HV ou e-nom = HV
   alors afficher "problème de fichier(s) vide(s)"
   sinon répéter
              si e-nom > m-nom
              alors (* seule la création est possible *)
                    si m-maj = "C"
                    alors enr-empmaj ← m-zones-communes
                          liremaj (mise-à-jour)
                          trait-empl (mise-à-jour,employé-maj,erreur)
                    sinon ecr-erreur (mise-à-jour,erreur)
                          liremaj (mise-à-jour)
                    fsi
              sinon si e-nom < m-nom
                     alors  (* employé non concerné par la maj *)
                       enr-empmaj ← enr-employé
                       écrire enr-empmaj
                       lireemp (employé)
```

```
                    sinon  (* employé existant à traiter *)
                       enr-empmaj ← enr-employé
                       trait-empl (mise-à-jour,employé-maj,erreur)
                       lireemp (employé)
                    fsi
                 fsi
        jqa e-nom = HV et m-nom = HV
     fsi
  fin


  procédure lireemp  (données modifées : employé)
  début
     lire enr-employé
     si fin-fichier
     alors e-nom ← HV
     fsi
  fin


  procédure liremaj  (données modifiées : mise-à-jour)
  début
     lire enr-maj
     si fin-fichier
     alors m-nom ← HV
     fsi
  fin


  procédure ecr-erreur  (données : mise-à-jour
                         données modifiées : erreur)
  début
     er-nom ← m-nom
     er-maj ← m-maj
     écrire enr-erreur
  fin


  procédure trait-empl  (données : mise-à-jour
                         données modifiées : employé-maj, erreur)
  var  supp : booléen
  début
      supp ← .faux.
      tant que m-nom = em-nom
          si m-maj = "M" ou m-maj = "S"
          alors si m-maj = "M"
                 alors em-fonction ← m-fonction
                 sinon supp ← .vrai.
                 fsi
          sinon ECR-ERREUR (mise-à-jour,erreur)
```

```
        fsi
        liremaj (mise-à-jour)
      ftq (* cas de m-nom = HV compris *)
      si non supp
      alors écrire enr-empmaj
      fsi
    fin
```

V – Mise à jour d'un fichier d'organisation séquentielle indexée

DEFINITION

Une seconde solution pour implanter les enregistrements sur le support est l'organisation séquentielle indexée. Les enregistrements sont rangés en respectant l'ordre croissant d'une des rubriques, appelée "clé", qui permet de repérer de façon unique un enregistrement. Le système de gestion de fichier gère les correspondances entre la place d'un enregistrement sur le support et la valeur de sa clé dans une table nommée TABLE D'INDEX.

Les seuls supports aptes à recevoir un fichier organisé en séquentiel indexé sont les SUPPORTS ADRESSABLES (disques, disquettes), l'adresse est la place de l'enregistrement.

La clé peut être soit une rubrique, soit un groupe de rubriques du fichier. Son choix doit avant tout assurer l'unicité d'un enregistrement pour une valeur de clé donnée.

L'organisation séquentielle indexée permet un ACCES DIRECT à un enregistrement précis, grâce à la table d'index mise à jour par le système de gestion de fichier, ou un ACCES SEQUENTIEL, similaire à celui effectué dans un fichier organisé en séquentiel.

DÉCLARATION ET INSTRUCTIONS

```
FICHIER <nom-fichier> (ORGANISATION INDEXEE ACCES SEQUENTIEL
                                            DIRECT
                       CLE = 'rubrique-x)
    ENREGISTREMENT <nom-enregistrement-fichier>
        <rubrique-1> : <type>
        <rubrique-2> : <type>
        .....
    FIN-ENREGISTREMENT
```

Instruction d'ouverture et de fermeture identiques à celles définies pour l'organisation séquentielle.

```
        (* ACCES DIRECT *)
    (* transfert de clé *)
rubrique-x ← valeur-cherchée
    (* lecture directe *)
```

```
LIRE <nom-enregistrement-fichier>
SI ERREUR-CLE
ALORS <traitement-1>  (* enregistrement inexistant *)
SINON <traitement-2>
FSI

        (* ACCES SEQUENTIEL *)

LIRE <nom-enregistrement-fichier>
SI FIN-FICHIER
ALORS <traitement-1>
SINON <traitement-2>
FSI

ECRIRE <nom-enregistrement-fichier>
SI ERREUR-CLE
ALORS <traitement-1> (* clé en double *)
SINON <traitement-2>
FSI

REECRIRE <nom-enregistrement-fichier>
SI ERREUR-CLE
ALORS <traitement-1> (* problème réécriture sur le disque, clé modifiée *)
SINON <traitement-2>
FSI

SUPPRIMER <nom-enregistrement-fichier>
SI ERREUR-CLE
ALORS <traitement-1> (* problème réécriture booléen système,
                        suppression logique *)
SINON <traitement-2>
FSI
```

ERREUR-CLE est un booléen du système de gestion de fichier. Le test de son contenu doit être effectué après toutes les actions exécutées sur le fichier, sans exception.

■ Algorithme – exercice 13

CREATION d'un fichier sur disque des produits en vente dans un magasin de peinture. Chaque produit possède un numéro qui l'identifie de façon unique, une désignation, une quantité en stock, un prix unitaire.

MÉTHODE

La méthode est identique à celle de création d'un fichier séquentiel.

- Déclaration du fichier produits.
- Saisie des valeurs.
- Validation de la saisie.
- Ecriture des informations de la mémoire sur le support.
- Question et reprise conditionnelle du traitement.

```
programme création
fichier produits(organisation indexée accès direct
                 clé = p-num)
    enregistrement enr-produits
        p-num      :  entier
        p-design   :  caractères
        p-stock    :  entier
        p-prix     :  entier
    fin-enregistrement
var  réponse, valid : caractères
début
   ouvrir produits (E)
   répéter
    afficher "donner le numéro de produit"
    saisir p-num
    afficher "donner la désignation"
    saisir p-design
    afficher "donner la quantité en stock"
    saisir p-stock
    afficher "donner le prix unitaire"
    saisir p-prix
        (* validation *)
    afficher "validez-vous les informations saisies (o/n) ?"
    saisir valid
    si  valid = "o"
    alors     (* écriture sur disque *)
        écrire enr-produits
        si erreur-clé
        alors afficher "écrire impossible, clé en double "
        fsi
    fsi
    afficher "autre produit à saisir (o/n) ?"
    saisir réponse
   jqa réponse = "n"
   fermer produits
fin
```

REMARQUE

Plusieurs systèmes de gestion de fichiers imposent à l'utilisateur de saisir les enregistrements dans l'ordre croissant des clés, lors de la création d'un fichier organisé en séquentiel indexé. Nous obtenons dans ce cas, l'affichage d'un message système dès qu'un enregistrement a une clé inférieure à la dernière valeur de clé traitée.

■ Algorithme – exercice 14

AFFICHAGE sur l'écran de toutes les informations contenues dans le fichier. Ce traitement nécessite un accès séquentiel, l'algorithme est donc identique à l'affichage des données d'un fichier séquentiel.

```
programme affichage
fichier produits(organisation indexée accès séquentiel
              clé = p-num)
    enregistrement enr-produits
        ....
    fin-enregistrement
début
   ouvrir produits (L)
   lire enr-produits
   si    fin-fichier
   alors afficher "le fichier des produits est vide"
   sinon afficher " fichier des produits    "
         afficher "numéro     désignation   stock  prix"
         répéter
              afficher p-num, p-design, p-stock, p-prix
              lire enr-produits
         jqa fin-fichier
   fsi
   fermer produits
fin
```

■ Algorithme – exercice 15

CONSULTATION des informations stockées sur un produit du fichier. Ce traitement est beaucoup plus rapide avec cette organisation, qui permet l'accès direct.

MÉTHODE

- Saisie du numéro de produit recherché.
- Lecture directe de cet enregistrement.
- Affichage des informations à consulter.

```
programme consultation
fichier produits (organisation indexée accès direct
              clé = p-num)
    enregistrement enr-produits
        p-num      :  entier
        p-design   :  caractères
        p-stock    :  entier
        p-prix     :  entier
    fin-enregistrement
var  numéro : entier
début
   ouvrir produits (L)
   afficher "donner le numéro du produit recherché"
   saisir numéro
     (* transfert valeur de la clé et lecture *)
   p-num ← numéro
   lire enr-produits
   si erreur-clé
   alors afficher "le produit n° ",numéro," n'existe pas"
```

```
   sinon afficher "produit n°          ",p-num
         afficher "désignation         ",p-design
         afficher "quantité en stock ",p-stock
         afficher "prix unitaire       ",p-prix
   fsi
   fermer produits
fin
```

■ Algorithme – exercice 16

MODIFICATION du prix unitaire d'un produit du fichier. Ce traitement utilise la même méthode que pour la modification d'un enregistrement d'un fichier séquentiel.

Deux différences :
– la procédure de recherche n'utilise pas d'itérative (elle est plus rapide d'exécution, grâce à l'accès direct);
– un test est nécessaire après l'instruction de réécriture.

```
programme modification
fichier produits(organisation indexée accès direct
                 clé = p-num)
     (* même déclaration que précédemment *)
var numéro : entier
   trouvé : booléen
début
   ouvrir produits (L/E)
   afficher "donner le numéro du produit qui change de prix "
   saisir numéro
   recherche (produits, numéro, trouvé)
   si trouvé
   alors afficher "modification du produit ",p-num,p-design
        afficher "ancien prix ",p-prix, "nouveau prix ? "
        saisir p-prix
        réécrire enr-produits
        si erreur-clé
        alors afficher "problème de réécriture sur disque ! "
        fsi
   sinon afficher "numéro : ",numéro," non trouvé dans le fichier"
   fsi
   fermer produits
fin

procédure recherche
   (données : produits, numéro
   résultat : trouvé)
début
   p-num ← numéro
   lire enr-produits
   si erreur clé
   alors trouvé ← .faux.
   sinon  trouvé ← .vrai.
   fsi
fin
```

La SUPPRESSION d'un enregistrement du fichier séquentiel indexé utilise le très simple sous-programme de recherche qui précède. Le programme principal, identique à celui de suppression dans un fichier séquentiel, effectue un test d'erreur de clé après la réécriture.

Les problèmes d'EDITIONS simples ou avec ruptures et d'APPAREILLAGES manipulant un fichier d'organisation séquentielle indexée utilisent l'accès séquentiel. Les algorithmes vus aux paragraphes III et IV de ce chapitre sont donc applicables à tous les fichiers, quelles que soient leurs organisations.

VI – Mise à jour d'un fichier d'organisation relative

DÉFINITION

Une troisième solution pour implanter les enregistrements sur le support est l'organisation directe. Elle gère l'emplacement d'un enregistrement, grâce à un calcul d'adresse à partir d'une valeur numérique de clé. La difficulté est d'obtenir une règle de calcul qui permette de gérer au mieux la place réservée sur le support, c'est-à-dire qu'un grand pourcentage de place doit être occupé et qu'un très faible pourcentage de valeurs de clé doit aboutir au même résultat d'adresse. Ce dernier cas est tout de même géré par le système de gestion de fichiers qui implante les enregistrements dont la valeur calculée d'adresse est déjà occupée, dans une zone nommée zone de débordement. Le chaînage des enregistrements écrits en zone de débordement est alors mis à jour par le système.

Un des cas particuliers de l'organisation directe est l'organisation relative, pour laquelle la règle de calcul est la plus simple de toutes. L'adresse est en correspondance directe avec la valeur de clé. Ainsi, pour un fichier implanté à partir de l'adresse 1BC2 et dont l'enregistrement a une longueur de 30 octets :

30 (en base 10) = 1E (en hexadécimal)

clé = 0 le premier enregistrement est à l'adresse			1BC2
clé = 1 le second	1BC2 + 1E	=	1BE0
clé = 2 le troisième	1BE0 + 1E	=	1BFE

L'organisation relative est intéressante pour des fichiers qui
– possèdent une clé numérique;
– dont les valeurs de cette clé appartiennent à un petit intervalle;
– dont la grande majorité des entiers de cet intervalle existent comme valeurs de clé d'enregistrement.

REMARQUE
– Les supports qui permettent l'organisation relative sont les supports adressables.
– L'organisation relative permet d'utiliser les deux accès connus, séquentiel et direct.
– La clé est une variable entière non signée, elle n'appartient pas à la liste des rubriques de l'enregistrement.

DÉCLARATION ET INSTRUCTIONS

```
FICHIER <nom-fichier>  (ORGANISATION RELATIVE ACCES SEQUENTIEL
                                                    DIRECT
                        CLE = variable-mémoire)
    ENREGISTREMENT <nom-enregistrement-fichier>
        <rubrique-1>
        <rubrique-2>
        ....
    FIN-ENREGISTREMENT
VAR <variable-mémoire> : ENTIER
```

Les instructions d'ouverture, de fermeture, de lecture, d'écriture, de réécriture et de suppression sont identiques à celles définies pour l'organisation séquentielle indexée. La suppression est toujours logique.

■ Algorithme – exercice 17

CREATION sur un support adressable d'un fichier d'organisation relative des représentants d'une société de vente à domicile. Les représentants sont numérotés séquentiellement, nous gérons pour chacun leur nom et prénom, leur adresse et leur date d'entrée dans la société.

Comment attribuer les numéros séquentiels des représentants ?
Lors de la première exécution de notre algorithme, le premier numéro à attribuer est le numéro 1, puis le 2, le 3, etc. Lors de la seconde exécution, nous devons retrouver le dernier numéro utilisé, perdu à la fin de la première exécution, car la mémoire est volatile. Nous devons donc l'écrire dans un fichier, pour le sauvegarder de façon durable. Le dernier numéro attribué à un représentant peut être stocké, soit dans un petit fichier indépendant, nommé "paramètre", soit dans un enregistrement spécial du fichier auquel il se rapporte. Nous choisissons cette deuxième solution. L'enregistrement de clé 0 va servir à mémoriser le dernier numéro attribué, par exemple dans la rubrique "r-datentr". Attention à ne pas traiter cet enregistrement comme un autre lors de l'utilisation séquentielle du fichier !

MÉTHODE (identique à celle de création d'un fichier séquentiel) :
– Déclaration du fichier représentant.
– Lecture de l'enregistrement de clé 0 et recherche du dernier numéro attribué ou initialisation de ce numéro à zéro.
– Saisie des informations.
– Validation des valeurs saisies.
– Ecriture des données de la mémoire sur le support.
– Question pour effectuer une autre saisie.
– Lecture de l'enregistrement de clé 0 et mise à jour du dernier numéro attribué ou écriture de cet enregistrement si inexistant.

```
programme création
fichier représentant  (organisation relative accès direct
                clé = numrep)
    enregistrement enr-représ
        r-nompren  :  caractères
        r-adr      :  caractères
        r-datentr  :  entier
```

```
    fin-enregistrement
var numrep, compteur : entiers
   réponse, valid : caractères
   fichier-vide : booléen
début
   ouvrir représentant (L/E)
        (* lecture du premier enregistrement "spécial" et
             recherche du dernier numéro attribué *)
   rech-dernum (représentant,numrep,compteur,fichier-vide)
        (* affectation de la rubrique date d'entrée *)
   r-datentr ← datsyst
   répéter
        (* calcul clé *)
     compteur ← compteur + 1
     numrep ← compteur
        (* saisie *)
     afficher "donner le nom et le prénom"
     saisir r-nompren
     afficher "donner l'adresse complète"
     saisir r-adr
     afficher "validez-vous les informations saisies (o/n) ?"
     saisir valid
     si  valid = "o"
     alors écrire enr-représ
           si erreur-clé
           alors afficher "problème d'écriture, clé : ", numrep
           fsi
     fsi
     afficher "autre représentant à saisir (o/n) ?"
     saisir réponse
   jqa réponse = "n"
        (* mise à jour du premier enregistrement *)
   maj-dernum (fichier-vide,compteur,représentant,numrep)
   fermer représentant
fin


procédure rech-dernum
   (données : représentant
   données modifiées : numrep
   résultat : compteur, fichier-vide)
début
   numrep ← 0
   lire enr-représ
   si   erreur-clé
   alors compteur ← 0
        fichier-vide ← .vrai.
   sinon compteur ← r-datentr
        fichier-vide ← .faux.
   fsi
fin
```

```
procédure maj-dernum
   (données : fichier-vide, compteur
   données modifiées : représentant, numrep)
début
   numrep ← 0
   si fichier-vide
   alors r-datentr ← compteur
        écrire enr-représ
        si    erreur-clé
        alors afficher "problème d'écriture clé 0 "
        fsi
   sinon
        lire enr-représ
        si erreur-clé
        alors afficher "problème seconde lecture clé 0 !!! "
        sinon r-datentr ← compteur
              réécrire enr-représ
              si erreur-clé
              alors afficher "problème réécriture clé 0 "
              fsi
        fsi
   fsi
fin
```

■ Algorithme – exercice 18

CONSULTATION d'un représentant

MÉTHODE (identique à la consultation d'un fichier séquentiel indexé) :
- Saisie du numéro de représentant recherché.
- Lecture directe de cet enregistrement.
- Affichage des informations à consulter.

```
programme consultation-relative
fichier représentant  (organisation relative accès direct
                       clé = numrep)
    enregistrement enr-représ
        r-nompren  :  caractères
        r-adr      :  caractères
        r-datentr  :  entier
    fin-enregistrement
var numrep : entier
début
   ouvrir représentant (L)
   afficher "donner le numéro de représentant recherché"
   saisir numrep
   lire enr-représ
   si erreur-clé
   alors afficher "le représentant n° ",numrep," n'existe pas"
```

```
        sinon afficher "représentant n°       ",numrep
              afficher "nom et prénom         ",r-nompren
              afficher "adresse         ",r-adr
              afficher "date d'entrée ",r-datentr
        fsi
        fermer représentant
    fin
```

L'AFFICHAGE, la MODIFICATION, la SUPPRESSION, l'EDITION et l'APPAREILLAGE utilisent des méthodes identiques aux traitements vus précédemment pour les fichiers d'organisation séquentielle indexée.

VII – Application 1 : gestion de bibliothèque

Soient les fichiers LIVRE et AUTEUR suivants.

livre org : seq. ind.
clé = l-nuliv

n° livre	l-nuliv
nom auteur	l-auteur
titre	l-titre
nombre exemplaires	l-nbex
nombre livres sortis	l-nbsort

auteur org : seq.ind.
clé = a-auteur

nom auteur	a-auteur
nombre livres écrits	a-nbliv
numéro des livres	a-nuliv (1:10) tableau

REMARQUE

Un livre acheté en plusieurs exemplaires ne figure qu'une fois dans le tableau.

■ Algorithme – exercice 19

MISE A JOUR des deux fichiers après l'achat de livres supplémentaires.

Dans le fichier "livre", nous pouvons augmenter le nombre d'exemplaires d'un livre existant, ou créer un enregistrement pour un nouveau livre. Dans le fichier "auteur", nous pouvons mettre à jour le tableau des numéros de livres écrits par un auteur, ou créer un enregistrement pour le premier livre d'un auteur.

MÉTHODE

– Saisie du numéro de livre et du nombre d'exemplaires achetés.
– Lecture directe dans le fichier livre.
– Si l'enregistrement existe
  Alors mise à jour du nombre d'exemplaires et réécriture livre
  Sinon saisie des informations complémentaires et écriture livre
    lecture directe dans le fichier auteur
    si l'enregistrement existe
    alors mise à jour du tableau et réécriture auteur
    sinon écriture auteur.
– Question à l'utilisateur pour effectuer une autre saisie.

```
programme saisie-livre
fichier livre (organisation indexée accès direct
        clé = l-nuliv )
    enregistrement enr-livre
        l-nuliv : entier
        l-auteur: caractères
        l-titre : caractères
        l-nbex  : entier
        l-nbsort: entier
    fin-enregistrement
fichier auteur  (organisation indexée accès direct
            clé = a-auteur)
    enregistrement enr-auteur
        a-auteur : caractères
        a-nbliv : entier
        a-nuliv(1:10) : tableau d'entiers
    fin-enregistrement
var numéro,nbexempl : entiers
  réponse : caractères
début
  ouvrir livre, auteur (L/E)
  répéter
    afficher "donner le numéro du livre"
    saisir numéro
    afficher "donner le nombre d'exemplaires achetés"
    saisir nbexempl
        (* lecture livre *)
    l-nuliv ← numéro
    lire enr-livre
    si  erreur-clé
    alors     (* ce livre n'est pas enregistré. Création *)
        création-livre (nbexempl, livre)
              (* lecture auteur *)
        a-auteur ← l-auteur
        lire enr-auteur
        si erreur-clé
        alors (* cet auteur n'est pas enregistré. Création *)
            création-auteur (numéro,auteur)
```

```
            sinon (* cet auteur est déjà enregistré. Mise à jour *)
                  maj-auteur (numéro,auteur)
            fsi
         sinon     (* ce livre est déjà enregistré. Mise à jour *)
            maj-livre (nbexempl, livre)
         fsi
         afficher "autre saisie (o/n) ? "
         saisir réponse
      jqa réponse = "n"
      fermer livre, auteur
fin


procédure création-livre (données : nbexempl
                          données modifiées : livre)
début
         (* saisie complémentaire *)
   afficher "donner le nom de l'auteur et le titre"
   saisir l-auteur, l-titre
   l-nbex ← nbexempl
   l-nbsort ← 0
         (* écriture livre *)
   écrire enr-livre
   si    erreur-clé
   alors afficher "problème écriture livre"
         (* la clé en double est improbable, vu la construction
           de l'algorithme *)
   fsi
fin


procédure création-auteur (données : numéro
                           données modifiées : auteur)
début
         (* affectation rubrique et écriture *)
   a-nbliv ← 1
   a-nuliv(1) ← numéro
   écrire enr-auteur
   si erreur-clé
   alors afficher "problème écriture auteur"
   fsi
fin


procédure maj-auteur  (données : numéro
                       données modifiées : auteur)
début
         (* mise à jour et réécriture *)
   si a-nbliv = 10
   alors afficher "plus de place pour cet auteur "
   sinon a-nbliv ← a-nbliv + 1
```

```
            a-nuliv(a-nbliv) ← numéro
            réécrire enr-auteur
            si    erreur-clé
            alors afficher "problème réécriture auteur"
            fsi
      fsi
fin


procédure maj-livre (données : nbexempl
                     données modifiées : livre)
début
         (* mise à jour et réécriture *)
      l-nbex ← l-nbex + nbexempl
      réécrire enr-livre
      si    erreur-clé
      alors afficher "problème réécriture livre "
      fsi
fin
```

■ Algorithme – exercice 20

CONSULTATION des titres, numéros, nombre d'exemplaires existants et sortis pour tous les livres d'un auteur.

MÉTHODE

- Saisir nom auteur recherché.
- Lecture directe du fichier auteur.
- Boucle de traitement de chaque livre de cet auteur
  - lecture directe du fichier livre
  - affichage des informations

```
programme consult-livre
fichier livre    (organisation indexée accès direct
        clé = l-nuliv)
fichier auteur   (organisation indexée accès direct
             clé = a-auteur)
var  nomauteur : caractères
début
   ouvrir livre, auteur (L)
   afficher "donner le nom de l'auteur à consulter"
   saisir nomauteur
     (* lecture auteur *)
   a-auteur ← nomauteur
   lire enr-auteur
   si erreur-clé
   alors afficher "auteur ",nomauteur, " inconnu"
   sinon       (* traitement de chaque livre *)
        afficher " n° livre - titre - nb.exempl - nb.sorti "
        pour cpt ← 1 jqa a-nbliv
```

```
                l-nuliv ← a-nuliv(cpt)
                lire enr-livre
                si erreur-clé
                alors afficher "livre ",a-nuliv(cpt)," non trouvé"
                sinon afficher l-nuliv, l-titre, l-nbex, l-nbsort
                fsi
            fpour
        fsi
        fermer livre, auteur
    fin
```

■ Algorithme – exercice 21

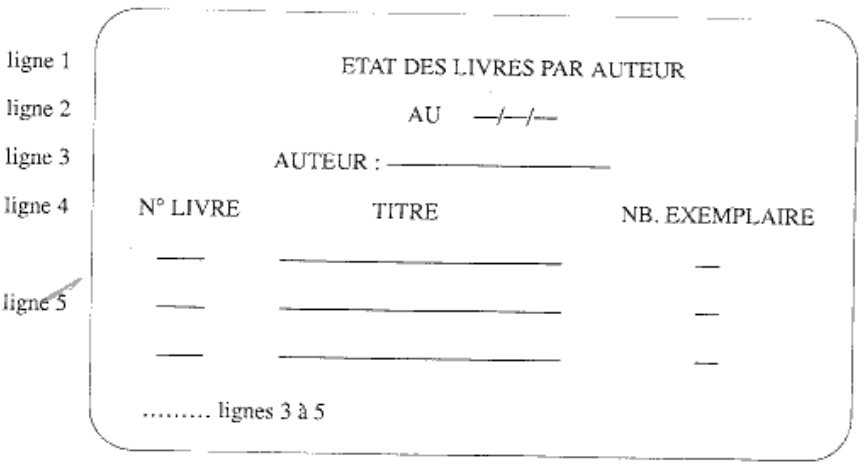
EDITION de tous les livres par auteur. Nous devons écrire un algorithme d'édition avec rupture et non reprendre l'algorithme de consultation et le rendre répétitif. L'édition utilise des accès séquentiels nettement plus rapides à exécuter que des accès directs sur tout le fichier livre.

Un tri du fichier livre sur le nom auteur croissant est l'étape préalable à notre algorithme. Le fichier résultant du tri étant séquentiel, il est une copie distincte du fichier "livre", nous le nommons "livre-trié".

MÉTHODE

1)– Dessin de l'état désiré.
- Numérotation des lignes.
- Groupement des lignes d'en-tête et de détail.

2) Algorithme d'édition avec une rupture.
- Déclaration des fichiers livre-trié, état et des lignes d'édition.
- Première lecture du fichier livre-trié.
- Ecriture des lignes d'en-tête.
- Boucle – écriture des lignes d'en-tête auteur
  - boucle – écriture des lignes détail
    - lecture du livre-trié suivant



```
programme édition-livre
fichier livre-trié (organisation séquentielle)
    enregistrement enr-livre-trié
        lt-nuliv   :  entier
        lt-auteur  :  caractères
        lt-titre   :  caractères
        lt-nbex    :  entier
        lt-nbsort  :  entier
    fin-enregistrement
fichier état (organisation séquentielle)
    enregistrement enr-état
        ligne :  caractères
    fin-enregistrement
const   ligne1, ligne4
var  l2-date, l5-nuliv, l5-nbex : entier
   auteur-traité, l3-auteur, l5-titre : caractères
   fini : booléen
début
   ouvrir livre-trié(L)
          état (E)
   lire enr-livre-trié
   si fin-fichier
   alors afficher "fichier livre vide, édition impossible"
   sinon (* édition *)
        fini ← .faux.
        l2-date ← datsyst
        écrire enr-état      (ligne1)
        écrire enr-état      (ligne2)
        répéter
             l3-auteur ← l-auteur
             écrire enr-état       (ligne3)
             écrire enr-état       (ligne4)
             auteur-traité ← l-auteur
             répéter
                l5-nuliv ← l-nuliv
                l5-titre ← l-titre
                l5-nbex ← l-nbex
                écrire enr-état    (ligne5)
                lire enr-livre-trié
                si fin-fichier
                alors fini ← .vrai.
                fsi
             jqa fini ou auteur-traité <> l-auteur
        jqa fini
        fsi
    fermer livre, état
fin
```

VIII – Application 2 : gestion de stock

Soient les fichiers PRODUIT, COMMANDE et CLIENT suivants :

Produit org : seq.ind.
clé = p-ref

référence produit	p-ref	
désignation	p-design	
quantité en stock	p-stock	
seuil de réapprovisionnement	p-seuil	
quantité minimale de lancement	p-lance	
code réapprovisionnement	p-codrea	"o" en cours de réapp. "n" sans réapp. prévu
date de réapprovisionnement	p-datrea	

Commande org : seq.

numéro de commande	c-numcom	
numéro de client	c-numcli	
référence produit	c-refprod (1:6)	minimum 1 produit maximum 6
quantité commandée	c-qtecom (1:6)	
quantité livrée	c-qteliv (1:6)	

Client org : seq.ind.
clé = cl-num

numéro de client	cl-num
nom et prénom	cl-nom
adresse	cl-adr
téléphone	cl-tel
date première commande	cl-datprem
date dernière commande	cl-datdern
nombre total de commandes	cl-totcom

(la rubrique cl-tel de l'enregistrement de clé "0" contient le dernier numéro de client attribué, utilisation décrite algorithme-exercice 17)

■ Algorithme – exercice 22

SAISIE des commandes.

Un test est effectué sur le numéro de client, et provoque la création d'un enregistrement dans le fichier client si besoin.

Nous n'effectuons pas de test sur l'existence des produits commandés, pour ne pas ralentir de façon considérable le déroulement de ce programme.

Un fichier "paramètre" indique le dernier numéro de commande utilisé. Ce fichier a un enregistrement unique. S'il est vide, nous devons le créer, sinon nous devons le mettre à jour.

fichier **paramètre** org : seq

dernier numéro de commande utilisé pa-dernum

MÉTHODE

– Déclaration des trois fichiers utilisés.
– Lecture du fichier paramètre et recherche du dernier numéro de commande attribué ou initialisation à 0 de ce numéro.
– Lecture du premier enregistrement du fichier client (clé = 0) et recherche du dernier numéro de client attribué ou initialisation à 0 de ce numéro.
– Boucle
  – incrément du numéro de commande
  – saisie du numéro de client
  – lecture directe du fichier client
    si le client n'est pas enregistré, saisie complémentaire et écriture
    sinon mise à jour et réécriture
  – boucle - saisie de la référence produit et de la quantité commandée
  – écriture du fichier commande
  – question à l'utilisateur pour effectuer une autre saisie de commande
– Lecture du fichier client (clé = 0), mise à jour et réécriture, ou écriture.
– Lecture du fichier paramètre et réécriture, ou écriture.

```
programme saisie
fichier commande (organisation séquentielle)
fichier client   (organisation indexée accès direct
                  clé = cl-num)
        (* contenu de l'enregistrement donné dans le texte *)
fichier paramètre  (organisation séquentielle)
    enregistrement enr-paramètre
        pa-dernum : entier
    fin-enregistrement
var cpt,numcom,numcli : entier
   autre : caractère
   param-vide,client-vide,arrêt : booléen
début
   ouvrir commande (E)
         client,paramètre (L/E)
             (* recherche des derniers numéros attribués *)
   recherche-dernum (paramètre,client,numcom,numcli
                     param-vide,client-vide)
```

```
        (* traitement des commandes *)
    répéter
      numcom ← numcom + 1
      c-numcom ← numcom
      afficher "donner le numéro de client"
      saisir c-numcli
      lire enr-client
      si  erreur-clé
      alors    création-client (c-numcli,client)
      sinon    maj-client (client)
      fsi
          (* traitement des produits commandés *)
      cpt ← 1
      répéter
          afficher "donner référence produit et quantité commandée "
          saisir c-refprod(cpt), c-qtecom(cpt)
          c-qteliv(cpt) ← 0
          cpt ← cpt +1
          si    cpt > 6
          alors arrêt ← .vrai.
          sinon afficher "autre produit commandé (O/N) ? "
                saisir autre
                arrêt ← autre = "n"
          fsi
      jqa arrêt
          (* écriture commande *)
      écrire enr-commande
      afficher "autre commande (O/N) ?"
      saisir autre
    jqa autre = "N"
        (* mise à jour des derniers numéros utilisés *)
    maj-dernum (numcom,numcli,param-vide,client-vide,paramètre,client)
    fermer commande, client, paramètre
fin


procédure recherche-dernum
    (données : paramètre, client
    résultats : numcom,numcli,param-vide,client-vide)
début
     (* lecture paramètre *)
    lire enr-paramètre
    si    fin-fichier
    alors numcom ← 0
          param-vide ← .vrai.
    sinon numcom ← pa-dernum
          param-vide ← .faux.
    fsi
      (* lecture client *)
    ci-num ← 0
    lire enr-client
```

```
    si    erreur-clé
    alors numcli ← 0
          client-vide ← .vrai.
    sinon numcli ← cl-tel
          client-vide ← .faux.
    fsi
fin


procédure maj-dernum
   (données : numcom,numcli,param-vide,client-vide
   données modifiées : paramètre, client)
début
   (* mise à jour paramètre. Pas de lecture nécessaire, car
     aucun autre enregistrement de ce fichier n'a été lu. *)
   pa-dernum ← numcom
   si param-vide
   alors écrire enr-paramètre
   sinon réécrire enr-paramètre
   fsi
   (* mise à jour client. lecture du premier enregistrement s'il existe *)
   cl-num ← 0
   si client-vide
   alors cl-tel ← numcli
         écrire enr-client
         si erreur-clé
         alors afficher "problème écriture client clé 0"
         fsi
   sinon
         lire enr-client
         si erreur-clé
         alors afficher "problème seconde lecture client clé 0"
         sinon cl-tel ← numcli
               réécrire enr-client
               si erreur-clé
               alors afficher "problème réécriture client clé 0"
               fsi
         fsi
   fsi
fin


procédure création-client
   (données : c-numcli
   données modifiées : client)
début
   cl-num ← c-numcli
   afficher "donner les nom et prénom "
   saisir cl-nom
   afficher "donner l'adresse et le numéro de téléphone "
   saisie cl-adr, cl-tel
   cl-datprem ← datsyst
```

```
        cl-datdern ← datsyst
        cl-totcom ← 1
        écrire enr-client
        si erreur-clé
        alors afficher "problème écriture client n° ", cl-num
        fsi
    fin


    procédure maj-client  (données modifiées : client)
    début
        cl-datdern ← datsyst
        cl-totcom ← cl-totcom + 1
        réécrire enr-client
        si erreur-clé
        alors afficher "problème réécriture client n° ", cl-num
        fsi
    fin
```

■ Algorithme – exercice 23

MISE A JOUR des fichiers commande et produit, lors de l'affectation des stocks disponibles aux différentes commandes et EDITION de la liste des produits en rupture de stock.

Nos règles de gestion sont les suivantes :
– une quantité commandée est livrée si elle est inférieure ou égale à la quantité en stock;
– une quantité commandée est livrée partiellement si elle est strictement supérieure à une quantité en stock non nulle;
– un produit, non en cours de réapprovisionnement, est en rupture de stock dès que sa quantité en stock est inférieure au seuil de réapprovisionnement;
– un produit en rupture de stock entraîne l'édition d'une ligne sur l'état et la mise à jour du code et de la date de réapprovisionnement.

L'état à fournir est le suivant :

PARIS LE —/—/—

LISTE DES PRODUITS EN RUPTURE DE STOCK

REFERENCE PRODUIT	DESIGNATION	UNITE LANCEMENT
_____	_____	____
_____	_____	____
_____	_____	____

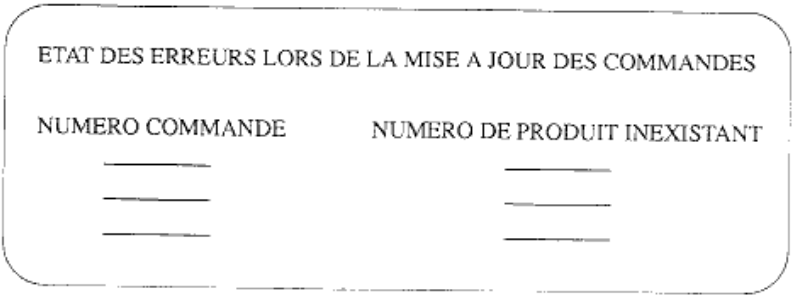
REMARQUE :

Nous supposons qu'il y a peu de produits en rupture de stock, il est donc préférable de

gérer l'édition pendant la mise à jour (les informations à éditer sont en mémoire pendant le traitement), et non après la mise à jour, car cela nécessite de réaliser par la suite la lecture du fichier produit dans son intégralité.

GESTION DES ERREURS

En cas de commande d'un produit inexistant, l'affichage d'un message à l'écran est une mauvaise solution, car les informations qui apparaissent sur un écran disparaissent définitivement dès que l'écran est plein. Nous devons gérer une édition durable des messages d'erreurs de ce type, dans un fichier d'impression nommé ERREUR. Sa gestion est identique à celle d'un fichier ETAT.



REMARQUE SUR LES PROBLEMES-SYSTEME LIÉS AUX FICHIERS

Nous effectuons souvent une écriture (création) après un ordre de lecture qui échoue, et une réécriture (mise à jour) après cette même instruction de lecture qui aboutit. La construction des algorithmes nous oblige à tester le cas d'erreur de clé après tous les ordres d'écriture ou de réécriture en accès direct. Or, dans les cas cités, le problème ne peut être "une clé en double" ou "un enregistrement inexistant". Nous rencontrons un problème plus grave, lié à la tête de lecture ou au support abîmé, qui doit entraîner l'arrêt immédiat du programme, dans le cas où l'utilisateur n'a pas la possibilité de stopper l'exécution par la saisie d'un caractère ou d'une touche de fonction.. Un booléen nommé "erreur-grave" peut être testé tout au long des répétitives, itératives et alternatives, mais nous préférons ne pas compliquer tous nos algorithmes, pour gérer un problème peu probable. Nous proposons d'utiliser une instruction ABANDON-PROG, qui rend la main à l'utilisateur en laissant les données traitées dans un état stable (fichiers fermés, etc.).

MÉTHODE

Le fichier directeur du traitement est le fichier commande qui est lu séquentiellement.
- Déclarations des quatre fichiers produit, commande, état et erreur.
- Première lecture du fichier commande.
- Ecriture des lignes d'en-tête des fichiers état et erreur.
- Boucle – pour chaque article commandé, lecture directe du fichier produit
  - si stock suffisant ou livraison partielle possible
    - mise à jour du stock
    - mise à jour de la quantité livrée
    - si problème de réapprovisionnement

* mise à jour du code et de la date de réapprovisionnement
* écriture d'une ligne détail sur l'état

• réécriture dans le fichier produit

– Réécriture dans le fichier commande.
– Lecture séquentielle du fichier commande.

```
programme maj-édition
(* détails des enregistrements donnés dans le texte *)
fichier produit (organisation indexée accès direct
                 clé = p-ref)
fichier commande (organisation séquentielle)
fichier état (organisation séquentielle)
    enregistrement enr-état
        ligne :  caractères
    fin-enregistrement
fichier erreur (organisation séquentielle)
    enregistrement enr-erreur
        lignebis:  caractères
    fin-enregistrement
const   ligne2,ligne3,lignebis1,lignebis2
var  cpt : entier
   arrêt : booléen
   l1-date,l4-ref,l4-lance : entier
   l4-design : caractères
   lb3-ref, lb3-com : entier
début
   ouvrir produit, commande (L/E)
          état, erreur(E)
   lire enr-commande
   si fin-fichier
   alors afficher "fichier commande vide, traitement inutile"
   sinon      (* en-tête *)
        l1-date ← datsyst
        écrire enr-état (ligne1, ligne2, ligne3)
        écrire enr-erreur (lignebis1, lignebis2)
             (* traitement des commandes *)
        répéter
             cpt ← 1
             (* traitement des produits commandés *)
             répéter
                p-ref ← c-refprod(cpt)
                lire enr-produit
                si erreur-clé
                alors lb3-ref ← c-refprod(cpt)
                      lb3-com ← c-refcom
                      écrire enr-erreur (lignebis3)
                sinon (* mise à jour rubriques *)
```

```
                traitement-stock-reapp (cpt,commande,
                                        produit,état,ligne4)
            fsi
            si cpt < 6
            alors cpt ← cpt + 1
                  arrêt ← c-refprod(cpt) = 0
            sinon arrêt ← .vrai.
            fsi
        jqa arrêt
            (* réécriture commande *)
        réécrire enr-commande
            (* lecture commande suivante *)
        lire enr-commande
     jqa fin-fichier
   fsi
   fermer produit, commande, état, erreur
fin


procédure traitement-stock-réapp    (données : cpt
    données modifiées : commande, produit, état, ligne4)
début
   si   p-stock > 0
   alors      (* maj. stock, livraison complète ou partielle *)
    si c-qtecom(cpt) < = p-stock
    alors p-stock ← p-stock - c-qtecom(cpt)
          c-qteliv(cpt) ← c-qtecom(cpt)
    sinon c-qteliv(cpt) ← p-stock
          p-stock ← 0
    fsi
            (* problème de réapprovisionnement *)
    si  p-stock < p-seuil et p-codrea = "n"
    alors l4-ref ← p-ref
          l4-design ← p-design
          l4-lance ← p-lance
          écrire enr-état (ligne4)
          p-codrea ← "o"
          p-datrea ← datsyst
    fsi
                 (* réécriture produit *)
              réécrire enr-produit
              si erreur-clé
              alors afficher "problème réécriture produit"
                    afficher "arrêt immédiat"
                    ABANDON-PROG
              fsi
  fsi
fin
```

IX – Application 3 : gestion de compte

Soient les fichiers CLIENT et REPRESENTANT suivants :

client org : seq. ind.
clé = c-cat + c-nom

catégorie client	c-cat	P :	particulier
		D :	détaillant
		G :	grossiste
nom du client	c-nom		
numéro du représentant	c-numrep		
solde	c-solde		
date dernière opération	c-datop		

représentant org : relative
clé = numéro-relatif

nom représentant	r-nom
adresse représentant	r-adr
cumul des pourcentages	r-cumul

Les numéros des représentants existaient avant l'informatisation des traitements, ils ont été attribués séquentiellement à partir du numéro 1001. L'analyste a conservé cette numérotation, mais, pour ne pas perdre de place sur le support avec le choix de l'organisation relative, la clé du fichier est égale au numéro de représentant moins mille.

■ Algorithme – exercice 24

MISE A JOUR des comptes clients et des pourcentages dus aux représentants.

La mise à jour du solde des clients est nécessaire à chaque réception de réglement de facture (incrément du solde), et à chaque enregistrement de commande (décrément du solde). Lors de la saisie d'un montant commandé, le représentant obtient

	3 %	de la commande d'un grossiste,
	5%	de la commande d'un détaillant,
et	8%	de la commande d'un particulier.

MÉTHODE

– Déclaration des deux fichiers.
– Saisie du montant traité et du sens de la mise à jour (débit ou crédit).
– Mise à jour du solde.
– Réécriture dans le fichier client.
– Si nous traitons une commande, lecture directe du fichier représentant, calcul du pourcentage, mise à jour du cumul et réécriture représentant.

```
programme maj-comptes
(* détails des enregistrements donnés dans le texte *)
fichier client  (organisation indexée accès direct
            clé = c-cat + c-nom)
fichier représentant  (organisation relative accès direct
              clé = numéro-relatif)
var numéro-relatif, montant : entier
   catégorie, nomcli, sens, réponse : caractères
début
   ouvrir     client, représentant (L/E)
   répéter
            (* saisie *)
    afficher "donner la catégorie et le nom du client"
    saisir catégorie, nomcli
            (* lecture client *)
    c-cat ← catégorie
    c-nom ← nomcli
    lire enr-client
    si  erreur-clé
    alors afficher "client inexistant"
    sinon afficher "donner le montant"
          saisir montant
          afficher "donner le sens du mouvement de compte
                   (D)ébit (C)rédit"
          saisir sens
          tant que sens <> "D" et sens <> "C"
             afficher "attention à la saisie du sens"
             saisir sens
          ftq
          si  sens = "C"
          alors c-solde ← c-solde + montant
          sinon c-solde ← c-solde - montant
                trait-pourcentage (catégorie,c-numrep,montant,
                                   numéro-relatif,représentant)
          fsi
          réécrire enr-client
          si  erreur-clé
          alors afficher "problème réécriture client "
               ABANDON-PROG
          fsi
    fsi
    afficher "autre saisie (oui/non) ? "
    saisir réponse
   jqa réponse = "non"
   fermer client, représentant
fin


procédure trait-pourcentage
   (données : catégorie, c-numrep, montant
   données modifiées : numéro-relatif,représentant)
```

```
var taux : entier
début
   numéro-relatif ← c-numrep - 1000
   lire enr-représentant
   si erreur-clé
   alors afficher "numéro représentant ", c-numrep, " inconnu "
        afficher "pourcentage non affecté ! "
   sinon      (* calcul pourcentage *)
        suivant catégorie faire
             "P"   :  taux ← 8
             "D"   :  taux ← 5
             "G"   :  taux ← 3
        finsuivant
        r-cumul ← r-cumul + montant * taux
             (* réécriture représentant *)
        réécrire enr-représentant
        si erreur-clé
        alors afficher "problème réécriture représentant "
             ABANDON-PROG
        fsi
   fsi
fin
```